

Java-Kompass mit GPX-Track

im Studiengang

Informatik

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Moritz Pfitzenmaier, Marius Müllmaier, Daniel Bensch

12.06.2026

Bearbeitungszeitraum: April 2026 – Juni 2026

Matrikelnummer: 8829906, 1341874, 5133713

Abstract

Richtung und Entfernung zu einem Ziel zu bestimmen, gehört zu den Grundaufgaben der mobilen Navigation. Die iOS-Anwendung *Kompass Professional* zeigt diese Peilung anschaulich, verzahnt ihre Berechnungslogik aber fest mit der Benutzeroberfläche und der Sensorhardware. Die vorliegende Arbeit löst diese Logik heraus und überführt sie in eine eigenständige Java-Bibliothek ohne Benutzeroberfläche.

Den Kern bildet die Berechnung von Azimut, Distanz und Himmelsrichtung aus WGS84-Koordinaten. Positions- und Kursdaten erhält die Bibliothek von einem Host-System; eigene Sensoren liest sie nicht. Über einen steuerbaren Session-Lebenszyklus lassen sich Tracks aufzeichnen, unterbrechen und beenden, wobei konfigurierbare Reduktionsverfahren die Datenmenge begrenzen. Den aufgezeichneten Track exportiert sie im Format GPX 1.1; optional bindet sie den Dienst What3Words an.

Neben der Implementierung als Maven-Projekt enthält die Arbeit eine Spezifikation nach IEEE 830. Alle Anforderungen sind nach den SOPHIST-Regeln formuliert und auf konkrete Testfälle rückführbar. Hinzu kommen nicht-funktionale Anforderungen, ein objektorientiertes Analyse- und Entwurfsmodell sowie automatisierte Tests, deren Bezug zu den Anforderungen eine Traceability-Matrix festhält.

Inhaltsverzeichnis

Abstract	i
Inhaltsverzeichnis	ii
Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
1 Einleitung	1
1.1 Zweck	1
1.2 Einsatzbereich	2
1.2.1 Problemstellung und fachlicher Hintergrund	2
1.2.2 Im Scope	2
1.2.3 Außerhalb des Scopes	2
1.3 Glossar	3
1.3.1 Fachbegriffe Navigation und Geografie	3
1.3.2 Begriffe zu Datenformaten und Protokollen	6
1.3.3 Begriffe zu Session und Systemverhalten	8
1.3.4 Begriffe zu Entwurfsmustern und Architektur	10
1.3.5 Begriffe zu Sicherheit	12
1.3.6 Begriffe zu Algorithmen	13
1.3.7 Begriffe zu Methodik und Qualitätssicherung	13
1.3.8 Begriffe zu Werkzeugen und Frameworks	15
1.4 Überblick	21
2 Allgemeine Beschreibung	22
2.1 Einbettung	22
2.1.1 Systemsicht und Systemgrenze	22
2.1.2 Schichtenarchitektur	22
2.1.3 Kommunikationsregeln	24
2.1.4 Physische Sicht	24
2.2 Funktionen	25
2.3 Benutzerprofile	27
2.3.1 Stakeholder und Einflussmatrix	27
2.3.2 Primärer Akteur: Host-Anwendung	27
2.3.3 Sekundärer Akteur: Externe Dienste	27
2.3.4 Funktionale Anforderungen – Übersicht	27
2.4 Einschränkungen	29
2.4.1 Im Scope	29
2.4.2 Außerhalb des Scopes	29
2.5 Annahmen und Abhängigkeiten	30

2.5.1	Annahmen	30
2.5.2	Abhängigkeiten	30
3	Spezifische Anforderungen	32
3.1	Funktionale Anforderungen	32
3.1.1	Akteure	32
3.2	Nicht-funktionale Anforderungen	53
3.3	Externe Schnittstellen	56
3.3.1	Öffentliche Java-API	56
3.3.2	Exception-Hierarchie	56
3.3.3	GPX 1.1-Ausgabeformat	57
3.4	Anforderungen an die Performance	58
3.4.1	Quantitative Leistungskennwerte	58
3.4.2	AD-01: Aktivitätsdiagramm onPositionUpdate	58
3.4.3	AD-02: Aktivitätsdiagramm GPX-Export	60
3.4.4	AD-03: Aktivitätsdiagramm Session starten	61
3.5	Qualitäts-Anforderungen	62
3.5.1	Qualitätsmodell nach ISO/IEC 25010	62
3.5.2	Produktdaten (/LD.../)	63
3.5.3	Testfälle (Traceability)	65
3.5.4	FMEA	67
4	Anhang	68
4.1	Erweiterte Akzeptanzkriterien (Kap. 3.1)	68
4.2	Haversine- und Azimutberechnung (Referenzalgorithmus)	69
4.3	Douglas–Peucker auf Kugeloberfläche (Hinweis für Erweiterungen)	70
4.4	Vollständige Traceability-Matrix	71
4.5	Objektorientiertes Analyse- und Entwurfsmodell (OOA/OOD)	72
4.5.1	OOA: Domänenmodell	72
4.5.2	OOD: Technischer Feinentwurf	72
4.5.3	Zustandsdiagramm: Session-Lebenszyklus	73
4.5.4	Subsystem-Dekomposition	74
4.6	Vorlesungs-Themen-Matrix (Abdeckung SWE)	76
4.7	Literatur- und Normenverweise	77
4.8	Glossar-Index (Schlagwort → Abschnitt)	78
4.9	Review-Checkliste „SOPHIST-Kriterien,, (Selbstaudit)	79
4.10	Versionshistorie	80

Abbildungsverzeichnis

Abbildung 1	Aktivitätsdiagramm /LF010/	34
Abbildung 2	Aktivitätsdiagramm /LF030/	36
Abbildung 3	Aktivitätsdiagramm /LF050/	38
Abbildung 4	Aktivitätsdiagramm /LF060/	39
Abbildung 5	Aktivitätsdiagramm /LF070/	41
Abbildung 6	Aktivitätsdiagramm /LF200/	43
Abbildung 7	Aktivitätsdiagramm /LF280/	45
Abbildung 8	Aktivitätsdiagramm /LF280/ (ohne Key)	46
Abbildung 9	Aktivitätsdiagramm /LF280/ (Timeout)	47
Abbildung 10	Aktivitätsdiagramm /LF230/	49
Abbildung 11	Aktivitätsdiagramm /LF080/	50
Abbildung 12	Aktivitätsdiagramm /LL160/	52
Abbildung 13	AD-01: Ablauf von <code>onPositionUpdate</code> (/LF030/) mit Validierungs-, Segment- und Budgetprüfung.	59
Abbildung 14	AD-02: Ablauf des GPX-Exports (/LF200/) mit optionaler Optimierer-Kette.	60
Abbildung 15	AD-03: Ablauf von <code>startSession</code> (/LF010/) mit Zustands- und Eingabeprüfung.	61
Abbildung 16	Zustandsdiagramm des Session-Lebenszyklus.	74

Tabellenverzeichnis

Tabelle 1	Externe Schnittstellen der Bibliothek und ihre Kommunikationsrichtungen.	22
Tabelle 2	Subsysteme und ihre Verantwortlichkeiten.	23
Tabelle 3	Erlaubte und verbotene Abhängigkeiten zwischen den Subsystemen.	24
Tabelle 4	Hauptfunktionsgruppen der Java-Peilungskomponente mit Verweis auf Anforderungen.	25
Tabelle 5	Entwurfsprinzipien und deren messbarer Nutzen.	26
Tabelle 6	Stakeholder der Java-Peilungskomponente mit Erwartungen und Einfluss.	27
Tabelle 7	Übersicht funktionaler Anforderungen der Java-Peilungskomponente (Traceability zu Kap. 3.1).	28
Tabelle 8	Übersicht der getroffenen Systemannahmen	30
Tabelle 9	Übersicht der Systemabhängigkeiten und externen Voraussetzungen	30
Tabelle 10	Primärakteure der Java-Peilungskomponente.	32
Tabelle 11	Sekundärakteure der Java-Peilungskomponente.	32
Tabelle 12	Vollständige öffentliche API-Methoden der Bibliothek.	56
Tabelle 13	Quantitative Performance-Anforderungen mit Messmethode.	58
Tabelle 14	Gewichtung der Qualitätsmerkmale nach ISO/IEC 25010 (Relevanzstufen je Unterkriterium).	62
Tabelle 15	Testfälle mit Traceability zu funktionalen und nicht-funktionalen Anforderungen.	65
Tabelle 16	FMEA: Fehlermodi, Folgen, Klassen und Gegenmaßnahmen.	67
Tabelle 17	Erweiterte Akzeptanzkriterien zu ausgewählten /LF.../- und /LL.../-Spezifikationen.	68
Tabelle 18	Vollständige Traceability-Matrix: Anforderung → OOA-Konzept → OOD-Klasse → Test.	71
Tabelle 19	Fachliches Domänenmodell (OOA): Konzepte und ihre Beziehungen.	72
Tabelle 20	OOD-Klassenübersicht: Zentrale Entwurfsklassen und Schnittstellen.	72
Tabelle 21	Zustandsdiagramm: Session-Lebenszyklus mit Transitionen und Guards.	74
Tabelle 22	Subsystem-Identifikation: Dienste und ihre Fehlerfälle.	74
Tabelle 23	Vorlesungsthemen und ihre Abdeckung durch Projektartefakte.	76
Tabelle 24	Primäre Normreferenzen und technische Quellen des Projekts.	77
Tabelle 25	Glossar-Schnellreferenz: Begriff und Fundstelle im Dokument.	78
Tabelle 26	Versionshistorie des Dokuments.	80

1 Einleitung

Dieses Kapitel klärt, welchen Zweck die Java-Peilungskomponente erfüllt, in welchem fachlichen Umfeld sie entsteht und wo ihre Grenzen liegen. Es benennt die Ziele der Bibliothek und die Kriterien, an denen sich ihr Erfolg messen lässt. Den Abschluss bildet eine Abgrenzung, welche Funktionen die Arbeit umsetzt und welche nicht. Diese Abgrenzung hält die Implementierung fokussiert und schafft Klarheit für alle, die die Bibliothek später einsetzen.

1.1 Zweck

Ziel dieses Dokuments ist eine vollständige und prüfbare Spezifikation einer Java-Bibliothek ohne Benutzeroberfläche, die Peilung, GPS-Track-Aufzeichnung und GPX-1.1-Export bereitstellt. Es richtet sich an zwei Gruppen: an Entwicklerinnen und Entwickler, die die Bibliothek in eigene Host-Anwendungen einbinden, und an Prüfende, die Anforderungen und Testabdeckung nachvollziehen. Jede Anforderung verweist auf konkrete Testfälle, sodass sich der Weg vom fachlichen Ziel bis zur Verifikation durchgehend belegen lässt.

Die Bibliothek entsteht in einer Lehrveranstaltung, soll aber auch praktisch brauchbar sein. Sie ist als eigenständige, wiederverwendbare Komponente angelegt, die sich ohne Anpassung in verschiedene Java-Umgebungen einbetten lässt — sei es eine Kommandozeilenanwendung, ein Hintergrunddienst oder eine serverseitige Verarbeitung.

Konkrete Ziele der Bibliothek:

- **Anforderungsanalyse und Spezifikation:** alle Anforderungen nach den SOPHIST-Regeln eindeutig und prüfbar formulieren — funktional, nicht-funktional, an den Datenschnittstellen und als Randbedingungen —, ergänzt um Qualitäts- und Risikobetrachtungen sowie ein objektorientiertes Analyse- und Entwurfsmodell.
- **Implementierung:** eine Java-Komponente ohne Benutzeroberfläche mit klaren Schnittstellen für Positions- und Kursdaten, konfigurierbarer Aufzeichnung, GPX-1.1-Export und optionaler What3Words-Anbindung.
- **Qualitätssicherung:** nachvollziehbare Testfälle je fachlichem Modul, automatisiert über Maven ausführbar und mit reproduzierbaren Ergebnissen als Grundlage für die Abnahme.
- **Lieferfähigkeit:** lauffähiger Quellcode — ohne ausführbares Fat-JAR als Hauptartefakt — samt einer Test-Suite, die sich direkt mit `mvn test` starten lässt.

Erfolgskriterien dieses Projekts:

- Die Komponente ist ohne UI lauffähig und per `mvn test` vollständig verifizierbar.
- Alle Anforderungen sind durch mindestens einen dokumentierten Testfall abdeckbar.
- GPX-Ausgaben enthalten den GPX-1.1-Namespace und valide Zeitstempel im UTC-ISO-8601-Format.
- Keine Klassen aus UI-Frameworks werden referenziert.

1.2 Einsatzbereich

1.2.1 Problemstellung und fachlicher Hintergrund

Als fachliche Referenz dient die iOS-Anwendung *Kompass Professional*. Sie demonstriert eine Peilungsfunktion: Die App zeigt an, in welcher Richtung und Entfernung ein Zielpunkt relativ zur aktuellen Position liegt — ohne Turn-by-Turn-Führung wie ein klassisches Navigationssystem. Dabei zeichnet sie einen GPS-Track auf, der sich als GPX exportieren lässt. Unter Peilung wird in dieser Arbeit demnach die Berechnung von Richtung und Entfernung von der aktuellen Position zu einem Ziel verstanden.

Problematisch ist dabei, dass die Peilungslogik der App fest mit ihrer Oberfläche und der Sensorhardware verzahnt ist und sich deshalb nicht eigenständig nutzen lässt. Genau hier setzt diese Arbeit an: Die Bibliothek soll Zielrichtung (Azimut), Entfernung und Ordinalrichtung aus WGS84-Koordinaten konsistent und reproduzierbar berechnen — unter klar definierten Eingangs- und Schnittstellenbedingungen. Die Verarbeitung läuft als Session mit Start, Laufzeit und Abbruch; auch nach einem Abbruch bleibt der bis dahin erfasste Track vollständig exportierbar.

Zielbild der Bibliothek: Aus den vom Host gelieferten Positions- und Kursdaten ermittelt die Bibliothek die Peilungsgrößen und zeichnet parallel einen GPS-Track auf, der sich als GPX 1.1 exportieren lässt. Optional lassen sich Koordinaten zusätzlich als What3Words-Adresse auflösen.

1.2.2 Im Scope

- Berechnung von Zielrichtung (Azimut), Entfernung und Ordinalrichtung auf WGS84-Basis, bei verfügbarem Kurswert auch der Kursabweichung.
- Klar definierte Integrationsschnittstellen, über die der Host Positionsdaten und zugehörige Metadaten (Zeitstempel, Geschwindigkeit, Genauigkeit) bereitstellt.
- Ein Session-Lebenszyklus mit Start, laufender Aufzeichnung und Abbruch. Nach einem Abbruch bleiben die bis dahin erfassten Track-Daten exportierbar.
- Konfigurierbare Aufzeichnung: Punktbudget (Soft-/Hard-Limit) und Segmentierung bei Zeitlücken. Wie oft Positionsupdates eintreffen, steuert der Host.
- Export der Track-Daten als GPX 1.1, wahlweise als String oder Bytefolge.
- Optional vor dem Export: erweiterbare Optimierungsverfahren (n-ter Punkt, Mindestabstand, Geraden-Heuristik, Douglas-Peucker) sowie eine What3Words-Auflösung mit Caching.

1.2.3 Außerhalb des Scopes

- Optimierung der eingehenden Rohdaten bereits beim Einlesen.
- Magnetische Peilung samt automatischer Deklinationskorrektur.
- Kartendarstellung, Map-Matching, Routing und Geocoding allgemeiner Adressen.
- Hardwareanbindung — der Host liefert fertige Messwerte.
- Feste Persistenzvorgaben wie ein vorgegebenes Dateiziel; über Speicherort und Dateiverwaltung entscheidet der Host.
- Cloud-Persistenz, Benutzer- und Rechteverwaltung.

1.3 Glossar

Das folgende Glossar definiert die zentralen Fachbegriffe dieses Dokuments einheitlich. Jede Begriffskarte folgt dabei demselben SOPHIST-Schema mit den Feldern Definition, Abgrenzung, Gültigkeit, Bezeichnung/Symbol und Quellverweis.

1.3.1 Fachbegriffe Navigation und Geografie

Begriff	Peilung (Bearing)
Definition	Bestimmung der horizontalen Zielrichtung von einem Bezugspunkt zu einem Zielpunkt.
Abgrenzung	Keine Turn-by-Turn-Navigation.
Gültigkeit	Gültig für WGS84-Koordinaten auf der Erdoberfläche.
Bezeichnung / Symbol	
Quellverweis	Peter Bohl

Begriff	Geografischer Azimut
Definition	Horizontaler Winkel zwischen der geografischen Nordrichtung und der Großkreisrichtung zum Ziel.
Abgrenzung	
Gültigkeit	Gültig für Entfernungen > 0 m.
Bezeichnung / Symbol	Symbol: α oder `azimuthDeg`; Einheit: ° (Grad).
Quellverweis	Gängige Navigationsliteratur.

Begriff	Kurs (Heading)
Definition	Orientierung der lokalen Horizontalebene relativ zu geografisch Nord, typischerweise als Winkel 0°–360°.
Abgrenzung	
Gültigkeit	Gültig, wenn der Host verlässliche Kursinformation liefert.
Bezeichnung / Symbol	Symbol: ψ oder `headingDeg`; Einheit: °.
Quellverweis	Kreisel-/Kompassmesstechnik.

Begriff	Kursabweichung (relative Bearing)
Definition	Winkel zwischen aktuellem Kurs und der Zielrichtung, zentriert typischerweise in $[-180^\circ, +180^\circ]$.
Abgrenzung	
Gültigkeit	
Bezeichnung / Symbol	Symbol: Δ oder `bearingErrorDeg`; Einheit: $^\circ$.
Quellverweis	Interne Systemdefinition gemäß Lastenheft /LF040/.

Begriff	Ordinalrichtung (Himmelsrichtung)
Definition	Vereinfachte 8er-Kompassangabe statt eines exakten Gradwerts.
Abgrenzung	
Gültigkeit	Wird aus dem Azimut abgeleitet und für eine leicht verständliche Richtungsanzeige genutzt.
Bezeichnung / Symbol	Werte: N, NE, E, SE, S, SW, W, NW.
Quellverweis	Kartographie; Anforderung /LF050/.

Begriff	Geografische Koordinate
Definition	Angabe eines Punktes auf der Erdoberfläche durch Breiten- und Längengrade.
Abgrenzung	
Gültigkeit	Gültig im WGS84-System.
Bezeichnung / Symbol	Notation: (lat, lon).
Quellverweis	ISO 19111

Begriff	Großkreisentfernung
Definition	Kürzeste Distanz zwischen zwei Ellipsoidpunkten entlang einer Großkreislinie auf der Kugelapproximation der Erde.
Abgrenzung	
Gültigkeit	Für kurze Distanzen < 20 km ist die sphärische Näherung üblicherweise ausreichend.
Bezeichnung / Symbol	Symbol: d oder `distanceM`; Einheit: m.
Quellverweis	Haversine-Formel als Referenznäherung.

Begriff	Haversine-Formel
Definition	Sphärische Formel zur Berechnung der Zentriwinkelspanne und daraus der Großkreisdistanz aus zwei Breiten-/Längengraden.
Abgrenzung	
Gültigkeit	Gültig für Kugelradius R.
Bezeichnung / Symbol	Implementationsreferenz: <code>`greatCircleDistanceMeters(lat1,lon1,lat2,lon2)`</code> .
Quellverweis	Wikipedia: Haversine-Formel

Begriff	ECEF (Earth-Centered, Earth-Fixed)
Definition	Rechtshändiges kartesisches Koordinatensystem mit Ursprung im Erdmittelpunkt (vereinfacht), Z-Achse durch den Nordpol.
Abgrenzung	
Gültigkeit	Interne Hilfskonstruktion für Distanzen im Raum optional; öffentliche API bleibt WGS84 lat/lon.
Bezeichnung / Symbol	Koordinaten X, Y, Z in Metern.
Quellverweis	ISO 19111; GNSS-Standardliteratur.

Begriff	EPSG:4326
Definition	Geographisches CRS: Achsenfolge Breite (φ), Länge (λ) in Grad auf WGS84-Ellipsoid.
Abgrenzung	
Gültigkeit	Gültig für alle Eingaben dieser Bibliothek; andere EPSG-Codes nur nach expliziter Erweiterung.
Bezeichnung / Symbol	CRS-Code als Metadaten-Hinweis exportierbar.
Quellverweis	EPSG Geodetic Parameter Dataset.

Begriff	Turn-by-Turn-Navigation
Definition	Schrittweise Routenführung entlang von Kanten eines Straßengraphen mit Manöveranweisungen.
Abgrenzung	
Gültigkeit	Explizit *nicht* Gegenstand der Peilungskomponente.
Bezeichnung / Symbol	—
Quellverweis	Allgemeiner Sprachgebrauch; Abgrenzung Projektauftrag.

1.3.2 Begriffe zu Datenformaten und Protokollen

Begriff	GPX (GPS Exchange Format)
Definition	XML-basiertes Austauschformat für Wegpunkte, Routen und Tracks.
Abgrenzung	
Gültigkeit	Version 1.1 ist verbindlich für dieses Projekt.
Bezeichnung / Symbol	
Quellverweis	Topografix GPX 1.1 Schema

Begriff	Trackpunkt
Definition	XML-Element innerhalb eines GPX-Tracks, das mindestens Breite, Länge und Zeit trägt.
Abgrenzung	
Gültigkeit	Gültig innerhalb eines Tracks.
Bezeichnung / Symbol	Attribute: `lat`, `lon`.
Quellverweis	GPX 1.1 Spezifikation

Begriff	HDOP (Horizontal Dilution of Precision)
Definition	Dimensionsloses Maß für die geometrische Güte der Satellitenkonstellation; niedriger ist typischerweise besser.
Abgrenzung	
Gültigkeit	Optional im Datenmodell und in GPX; keine automatische Verwerfung beim Einlesen (/LF130/ Out-of-Scope).
Bezeichnung / Symbol	
Quellverweis	Herstellerdokumentation GNSS-Empfänger.

Begriff	UTC und `Instant`
Definition	UTC ist die einheitliche Zeitbasis ohne Sommerzeit; `java.time.Instant` beschreibt einen exakten Zeitpunkt auf dieser Skala.
Abgrenzung	
Gültigkeit	Interne Zeitstempel laufen durchgängig über `Instant`; Ausgabe nach GPX erfolgt als ISO-8601 in UTC.
Bezeichnung / Symbol	Im Code: `ClockPort.instant()` und `DateTimeFormatter.ISO_INSTANT`.
Quellverweis	ISO 8601; Java SE Date-Time-Package.

Begriff	UTF-8
Definition	Byte-Kodierung für Unicode-Zeichen; verbindlich für GPX-XML in diesem Projekt.
Abgrenzung	
Gültigkeit	Alle serialisierten GPX-Dokumente.
Bezeichnung / Symbol	Kennung in XML-Deklaration optional, Bytes konsistent.
Quellverweis	Unicode Standard Annex #15.

1.3.3 Begriffe zu Session und Systemverhalten

Begriff	Session
Definition	Eine Session ist eine zeitlich begrenzte Instanz zur Speicherung von Zustandsinformationen eines Benutzers oder Prozesses.
Abgrenzung	Abzugrenzen ist die Session von einer einzelnen Anfrage sowie von persistenten Objekten, die dauerhaft gespeichert werden. Eine Session ist temporär und zustandsbehaftet.
Gültigkeit	Die Gültigkeit einer Session erstreckt sich über die Dauer der Interaktion und endet durch Timeout.
Bezeichnung / Symbol	Session-ID
Quellverweis	Lehrmaterial Softwareentwicklung

Begriff	Abbruchverhalten (Abort)
Definition	Vorzeitiges Beenden einer aktiven Session durch den Host, ohne die bis dahin erfassten Daten zu verlieren.
Abgrenzung	
Gültigkeit	Session wechselt auf `ABORTED`; Rückgabe erfolgt als `GpxResult` (UTF-8-Bytes plus String-Zugriff) gemäß /LF070/ und /LF230/.
Bezeichnung / Symbol	Persistenz beim Abort erfolgt nur bei aktivem `persistOnAbort`.
Quellverweis	Klärungsgespräch; /LF070/

Begriff	Observer / Listener
Definition	Benachrichtigen registrierte Beobachter über Zustandsänderungen.
Abgrenzung	
Gültigkeit	Synchronität gemäß Konfiguration /LF080/.
Bezeichnung / Symbol	Java-Pattern: `java.util.EventListener`-Familie.
Quellverweis	/LF080/

Begriff	Trackoptimierung
Definition	Austauschbare Algorithmen hinter gemeinsamer Schnittstelle zur Reduktion oder Vereinfachung von Trackpunkten.
Abgrenzung	
Gültigkeit	Nur optional vor GPX-Export über `SessionConfig.addOptimizer` (leere Liste = unveränderter Rohtrack); /LF240/-/LF270/.
Bezeichnung / Symbol	
Quellverweis	/LF480/

Begriff	Soft-Limit / Hard-Limit (Punktbudget)
Definition	Zweistufige Begrenzung der gespeicherten Trackpunkte: erst Warnung, dann harter Stopp.
Abgrenzung	
Gültigkeit	Konfigurierbar über `softLimitPoints` und `hardLimitPoints`; `0` bedeutet deaktiviert (/LF120/, /LF180/).
Bezeichnung / Symbol	Code: `SOFT_LIMIT_WARN`, `HARD_LIMIT_REACHED`.
Quellverweis	Ressourcen-Schutz in eingebetteten Systemen; projektspezifisch.

Begriff	Overflow-Modus (Punktbudget)
Definition	Definiertes Verhalten bei Erreichen des Hard-Limits: Stopp der Aufzeichnung vs. Downsampling.
Abgrenzung	
Gültigkeit	Konfigurationsabhängig /LF180/.
Bezeichnung / Symbol	Enum-Kandidat: `STOP`, `DOWNSAMPLE` (Beispiel).
Quellverweis	Interne Spezifikation.

1.3.4 Begriffe zu Entwurfsmustern und Architektur

Begriff	API (Application Programming Interface)
Definition	Programmierschnittstelle, über die externe Systeme mit der Peilungskomponente interagieren.
Abgrenzung	
Gültigkeit	Öffentliche Methoden der Bibliothek.
Bezeichnung / Symbol	
Quellverweis	Software Engineering Standardbegriff

Begriff	Builder (Konfigurationsaufbau)
Definition	Muster, mit dem man eine Konfiguration Schritt für Schritt baut statt mit einem langen Konstruktor.
Abgrenzung	
Gültigkeit	Im Projekt über `SessionConfig.builder()`, beim Start werden die relevanten Parameter als `RecordingParameters` eingefroren (/LF410/).
Bezeichnung / Symbol	Beispiel: `SessionConfig.builder().segmentGapThreshold(java.time.Duration.ofMinutes(5)).build()`
Quellverweis	Design Patterns

Begriff	Immutability
Definition	Ein Objekt bleibt nach der Erzeugung unverändert; für Änderungen wird ein neues Objekt erstellt.
Abgrenzung	
Gültigkeit	Trifft hier u.\ a. auf `SessionConfig` und `GpsPoint` zu.
Bezeichnung / Symbol	Hilft bei Nebenläufigkeit, weil Leser keine veränderbaren Zwischenzustände sehen.
Quellverweis	Software Engineering Best Practices

Begriff	Thread-Sicherheit
Definition	Eigenschaft eines Systems, bei paralleler Ausführung durch mehrere Threads korrekt zu funktionieren.
Abgrenzung	
Gültigkeit	Relevant für Observer, Immutability und Eventverarbeitung.
Bezeichnung / Symbol	
Quellverweis	Nebenläufigkeit in Softwaresystemen

Begriff	Semantische Exception
Definition	Fehlerklasse mit maschinenlesbarem `errorCode` und kontextualisierter Message.
Abgrenzung	
Gültigkeit	Öffentliche API der Bibliothek.
Bezeichnung / Symbol	Feld: `String code`.
Quellverweis	/LF090/; Best Practice API Design.

1.3.5 Begriffe zu Sicherheit

Begriff	Path Traversal
Definition	Sicherheitsproblem, bei dem ein Pfad absichtlich aus dem erlaubten Zielordner ausbricht (z.\ B. mit `..\`).
Abgrenzung	
Gültigkeit	Beim Export verpflichtend zu verhindern (/LF320/).
Bezeichnung / Symbol	Umsetzung in `SafeFileSink`: Pfad normalisieren und gegen erlaubtes Basisverzeichnis prüfen.
Quellverweis	OWASP Path Traversal

Begriff	XXE (XML External Entity)
Definition	Angriffsklasse beim XML-Parsing über externe Entities.
Abgrenzung	
Gültigkeit	Die Komponente erzeugt XML selbst und escaped Inhalte (/LL130/); das XXE-Risiko liegt vor allem bei fremden Parsern, die diese XML später lesen.
Bezeichnung / Symbol	Empfehlung: DTD/External-Entity-Verarbeitung im konsumierenden Parser deaktivieren.
Quellverweis	W3C XML Recommendation

Begriff	Atomares Schreiben
Definition	Datei wird zuerst temporär geschrieben und dann in den Zielpfad verschoben, um halbfertige Dateien zu vermeiden.
Abgrenzung	
Gültigkeit	Wichtig für die Konsistenz bei Exporten.
Bezeichnung / Symbol	Temporäre Datei (`*.tmp`).
Quellverweis	Anforderung /LF220/.

1.3.6 Begriffe zu Algorithmen

Begriff	Douglas–Peucker-Algorithmus
Definition	Algorithmus zur Linienvereinfachung: Punkte mit geringer Abweichung von der Verbindungslinie werden rekursiv entfernt.
Abgrenzung	
Gültigkeit	Im Projekt mit metrischer Toleranz in Metern und lokaler Tangentialebene als Näherung (/LF270/).
Bezeichnung / Symbol	Parameter im Code: `epsilonM`.
Quellverweis	D. Douglas, T. Peucker, Algorithms for the reduction of the number of points required to represent a digitized line or its caricature, 1973.

Begriff	Kollinearitätsreduktion
Definition	Reduktion fast kollinearer Punktfolgen auf Start- und Endpunkt innerhalb eines Winkel-/Seitenbandes.
Abgrenzung	
Gültigkeit	Explizit keine Garantie für enge Kurvenradien /LF260/.
Bezeichnung / Symbol	Heuristikparameter: Toleranzband.
Quellverweis	Klärungsgespräch Peter Bohl; /LF260/.

1.3.7 Begriffe zu Methodik und Qualitätssicherung

Begriff	Anforderung
Definition	Beschriebene Eigenschaft oder Fähigkeit, die ein System erfüllen muss oder soll.
Abgrenzung	
Gültigkeit	Gültig für alle /LF.../, /LL.../, /LD.../ Artefakte.
Bezeichnung / Symbol	Typen: funktional, nicht-funktional.
Quellverweis	ISO/IEC/IEEE 29148.

Begriff	SOPHIST-Regeln
Definition	Regelsatz, um Anforderungen klar, überprüfbar und ohne Widersprüche zu formulieren.
Abgrenzung	
Gültigkeit	Dient als Qualitätsmaßstab für die Formulierung der `/LF.../`, `/LL.../` und `/LD.../`-Artefakte.
Bezeichnung / Symbol	Wird in Reviews als kompakte Checkliste eingesetzt.
Quellverweis	Software Engineering Vorlesung, SOPHIST-Regeln

Begriff	Lastenheft vs. Pflichtenheft
Definition	Lastenheft: fachliche, wirtschaftliche und politische Zielsetzung aus Auftraggebersicht. Pflichtenheft: technische Umsetzungsvorgaben aus Entwicklersicht.
Abgrenzung	
Gültigkeit	In dieser Arbeit: Kapitel „Allgemeine Beschreibung“ ≈ Lastenheft-Teile; Kapitel „Spezifische Anforderungen“ ≈ Pflichtenheft.
Bezeichnung / Symbol	IEEE 830-1998
Quellverweis	Software Engineering Vorlesung

Begriff	Traceability (Rückverfolgbarkeit)
Definition	Nachvollziehbare Verbindung zwischen Anforderung, Design, Code und Test.
Abgrenzung	
Gültigkeit	Wichtig über den gesamten Lebenszyklus, besonders bei `/LF.../` und `/TC.../`.
Bezeichnung / Symbol	Im Projekt als Matrix in `implementation/TRACEABILITY.md`.
Quellverweis	Software Engineering Vorlesung

Begriff	Property "deterministisch"
Definition	Gleiche Eingaben und gleiche konfigurierte Abhängigkeiten (Clock) erzeugen identische Ausgaben.
Abgrenzung	
Gültigkeit	Gilt für Kern-Pfade ohne bewusst randomisierte Algorithmen.
Bezeichnung / Symbol	—
Quellverweis	/LL190/, /LF340/.

1.3.8 Begriffe zu Werkzeugen und Frameworks

Begriff	JVM (Java Virtual Machine)
Definition	Laufzeitumgebung, die Java-Bytecode plattformunabhängig ausführt; die Bibliothek setzt eine JVM voraus, stellt selbst jedoch keine bereit.
Abgrenzung	
Gültigkeit	Ziel-Kompatibilität: Java 11 (LTS); die Bibliothek wird als reines Source-Artefakt ohne ausführbares JAR ausgeliefert.
Bezeichnung / Symbol	Die JVM-Version des Host-Projekts muss ≥ 11 sein; niedrigere Versionen werden nicht unterstützt.
Quellverweis	Systemvoraussetzungen; /LF000/

Begriff	Maven
Definition	Standardwerkzeug für Build, Testlauf und Abhängigkeitsverwaltung in Java-Projekten.
Abgrenzung	
Gültigkeit	Im Projekt zentraler Einstieg für Build und Tests (<code>`mvn test`</code> , /LF450/).
Bezeichnung / Symbol	Artefakte werden über <code>`groupId:artifactId:version`</code> aufgelöst.
Quellverweis	Apache Maven Project Dokumentation, Software Engineering Vorlesung

Begriff	Maven-Profil (`w3w`)
Definition	Optionale Build-Konfiguration, die What3Words-Clientabhängigkeiten aktiviert.
Abgrenzung	
Gültigkeit	Standardbuild ohne Profil bleibt offline-fähig.
Bezeichnung / Symbol	Aktivierung: `mvn -Pw3w test` (konzeptionell).
Quellverweis	Apache Maven Documentation – Profiles.

Begriff	SLF4J (Simple Logging Facade for Java)
Definition	Einheitliche Logging-Schnittstelle, die konkrete Logger-Backends austauschbar macht.
Abgrenzung	
Gültigkeit	Die Komponente bindet Logging über den Adapter `Slf4jLoggerAdapter` ein (/LL140/).
Bezeichnung / Symbol	Typischer Einstieg: `LoggerFactory.getLogger(...)`.
Quellverweis	https://www.slf4j.org/

Begriff	JUnit 5
Definition	Modernes Java-Testframework für Unit- und Integrationsnahe Tests.
Abgrenzung	
Gültigkeit	In diesem Projekt Grundlage der Modul-Tests; aktuell vor allem mit `@Test`.
Bezeichnung / Symbol	Bietet u.\ a. Assertions, Lifecycle-Hooks und Erweiterungen.
Quellverweis	https://junit.org/junit5/

Begriff	JaCoCo
Definition	Code-Coverage-Tool für Java, Integration über Maven-Plugin.
Abgrenzung	
Gültigkeit	Messgröße für `/LL150/`.
Bezeichnung / Symbol	Report: `target/site/jacoco/index.html`.
Quellverweis	https://www.jacoco.org/jacoco/trunk/doc/

Begriff	Checkstyle
Definition	Werkzeug für statische Stil- und Strukturprüfungen im Java-Code.
Abgrenzung	
Gültigkeit	Im Build integriert; die aktuelle Projektkonfiguration ist bewusst schlank gehalten.
Bezeichnung / Symbol	Konfiguration in `implementation/checkstyle.xml`.
Quellverweis	https://checkstyle.org/

Begriff	ArchUnit
Definition	Bibliothek zur Überprüfung architektonischer Regeln im JUnit-Test (z.\ B. keine AWT-Imports).
Abgrenzung	
Gültigkeit	Optional zur Absicherung von `/LF430/`.
Bezeichnung / Symbol	Regel: `noClasses().that()...`
Quellverweis	https://www.archunit.org/

Begriff	Jimfs
Definition	In-Memory-Dateisystem-Implementierung für JDK `java.nio.file`, geeignet für Tests.
Abgrenzung	
Gültigkeit	Nur Testscope.
Bezeichnung / Symbol	Google Jimfs Projekt.
Quellverweis	https://github.com/google/jimfs

Begriff	Mutationstest (PIT)
Definition	Testtechnik, bei der kleine Codeänderungen (Mutanten) erzeugt werden, um die Schärfe der Tests zu prüfen.
Abgrenzung	
Gültigkeit	Als Qualitätsvertiefung möglich, aber nicht Teil des Standard-Builds.
Bezeichnung / Symbol	Kennzahl: Mutations-Score.
Quellverweis	Software Engineering Vorlesung

Begriff	Property-Based Testing (jqwik)
Definition	Testansatz, bei dem allgemeine Eigenschaften statt einzelner Beispiele geprüft werden.
Abgrenzung	
Gültigkeit	Sinnvoll für Invarianten wie Distanzsymmetrie oder Grenzfälle bei Winkeln.
Bezeichnung / Symbol	Kann optional mit jqwik umgesetzt werden.
Quellverweis	https://jqwik.net/

Begriff	Referenzimplementierung (Test)
Definition	Unabhängige, minimal gehaltene Berechnung derselben Größe zum Abgleich von Toleranzen.
Abgrenzung	
Gültigkeit	Testcode, nicht Produktivpfad.
Bezeichnung / Symbol	`ReferenceHaversine` (Bezeichner frei wählbar).
Quellverweis	/LL020/ Testpraxis.

Begriff	What3Words (W3W)
Definition	Dienst, der Koordinaten in ein Drei-Wort-Format umwandelt.
Abgrenzung	
Gültigkeit	Optionaler Teil des Systems (/LF280/); Ergebnisse werden gecacht (/LF290/).
Bezeichnung / Symbol	
Quellverweis	what3words Limited, Peter Bohl

Begriff	System (Softwarekomponente)
Definition	Abgegrenzter Teil eines Softwaresystems mit klar definierten Schnittstellen und Verantwortlichkeiten.
Abgrenzung	
Gültigkeit	Hier: die Peilungskomponente als eigenständige Bibliothek.
Bezeichnung / Symbol	
Quellverweis	Softwarearchitektur-Grundlagen

Begriff	Ist-/Soll-Vergleich (Referenz-App)
Definition	Qualitative Gegenüberstellung der Referenz-App mit der Java-Bibliothek ohne pixelgenaue UI-Nachbildung.
Abgrenzung	
Gültigkeit	Dient der Motivation und Begriffsableitung, nicht als normative Spezifikation der UI.
Bezeichnung / Symbol	—
Quellverweis	Apple App Store, Produktseite Kompass Professional.

Begriff	Genauigkeit
Definition	Accuracy beschreibt die Nähe zum wahren Wert, Precision die Wiederholgenauigkeit von Messungen.
Abgrenzung	
Gültigkeit	Relevant für GPS-Datenbewertung.
Bezeichnung / Symbol	
Quellverweis	Messtechnik-Grundlagen

1.4 Überblick

Das Dokument folgt dem Aufbau nach IEEE 830 und gliedert sich in drei Hauptkapitel:

Kapitel 1 – Einleitung legt Zweck und Einsatzbereich der Bibliothek fest, vereinheitlicht die Terminologie im Glossar und gibt einen Überblick über den Dokumentaufbau.

Kapitel 2 – Allgemeine Beschreibung zeigt, wie sich die Bibliothek in ihr Umfeld einfügt (System- und physische Sicht, externe Kommunikationspartner), benennt die Hauptfunktionen und hält systemweite Einschränkungen, Annahmen und Abhängigkeiten fest.

Kapitel 3 – Spezifische Anforderungen bildet den normativen Kern: funktionale Spezifikationen (/LF.../) mit Aktivitätsdiagrammen (Kap. 3.1), nicht-funktionale Anforderungen (/LL.../), externe Schnittstellen, Performance-Anforderungen sowie das Qualitätsmodell nach ISO/IEC 25010.

Der **Anhang** ergänzt das Ganze um Traceability-Matrizen, das objektorientierte Analyse- und Entwurfsmodell, Review-Checklisten, Normreferenzen, erweiterte Akzeptanzkriterien und algorithmische Hinweise.

2 Allgemeine Beschreibung

Dieses Kapitel betrachtet die Java-Peilungskomponente aus der Übersicht: wie sie sich in ihr technisches Umfeld einfügt, welche Hauptfunktionen sie bietet, welche Stakeholder und Benutzerprofile eine Rolle spielen und welche systemweiten Einschränkungen, Annahmen und Abhängigkeiten gelten. Es entspricht dem Abschnitt „General Description“, der IEEE-830-Vorlage.

2.1 Einbettung

2.1.1 Systemsicht und Systemgrenze

Die Java-Peilungskomponente ist eine reine Bibliothek ohne eigene Benutzeroberfläche. Sie wird in den Prozess einer **Host-Anwendung** eingebettet und läuft in deren JVM. Es existiert kein eigener Prozess, kein eigenständiger Server und kein Nachrichtensystem.

Die Systemgrenze umschließt ausschließlich die Java-Peilungskomponente. Sensordaten (GNSS, Kompass) werden nicht direkt gelesen. Die Host-Anwendung ist verantwortlich für Sensorfusion und Gerätezugriff und übergibt fertige Messwerte über die öffentliche API.

Externe Kommunikation findet ausschließlich über zwei optionale Kanäle statt: HTTPS zur W3W-API und Dateisystemzugriffe für den GPX-Export. Beide Kanäle sind abschaltbar; die Kernfunktion (Peilung, Track-Aufzeichnung) läuft ohne Netzwerkverbindung.

Externe Schnittstelle	Richtung	Zweck
Host-Anwendung	eingehend	Steuerung des Session-Lebenszyklus; Übergabe von Positions- und Kursdaten
Listener / Result	ausgehend	Snapshots, Statuswechsel, Fehlerereignisse; GPX-Rückgabe an den Host
Dateisystem	ausgehend, optional	GPX-Persistenz nur bei expliziter Konfiguration durch den Host
W3W-HTTP	ausgehend, optional	Reverse-Lookup mit lokalem Cache; ohne Netzwerk vollständig deaktivierbar

Tabelle 1: Externe Schnittstellen der Bibliothek und ihre Kommunikationsrichtungen.

2.1.2 Schichtenarchitektur

Das System ist vertikal in vier Subsysteme gegliedert, die als streng gerichtete Schichten angeordnet sind. Jede Schicht kommuniziert ausschließlich mit der direkt darunterliegenden; ein Überspringen von Schichten ist verboten. Dadurch entstehen keine zyklischen Abhängigkeiten und jede Schicht ist einzeln testbar und austauschbar.

Subsystem	Kernverantwortung	Bereitgestellte Dienste
API / Application Service	Use-Case-Orchestrierung; Session-Verwaltung; Fehlerbehandlung	Session starten, beenden und abbrechen; konsistenten Zustandsschnappschuss liefern
Domain Core	Fachlogik ohne I/O-Abhängigkeiten	Azimut, Distanz, Ordinalrichtung; Rohspeicher, Validierung, Segmentierung, Export-Optimierer
Ports	Abstraktion technischer Abhängigkeiten	Stabile Schnittstellen für GPX, Zeitgeber, Dateisystem, Logging und W3W
Infrastruktur / Adapter	Konkrete Implementierung der Ports	XML-Serialisierung, Datei-I/O, HTTP-Client, Systemuhr, Logging-Backend

Tabelle 2: Subsysteme und ihre Verantwortlichkeiten.

Schichten von oben nach unten

Host-System (außerhalb der Komponente)

Beliebige Java-Anwendung (Desktop, Web, Embedded). Greift ausschließlich über die öffentliche API-Fassade zu. Keine direkten Infrastrukturzugriffe.

↓ nur über öffentliche API-Aufrufe

API / Application Service

Steuert den Session-Lebenszyklus, prüft Vorbedingungen, bildet Fehler auf semantische Codes ab und orchestriert die Fachfälle. Kennt die Domain, kennt keine Adapter.

↓ nur über Domain-Schnittstellen

Domain Core

Enthält alle fachlichen Regeln und Berechnungen (Azimut, Distanz, Rohspeicher, Segmentierung, Punktbudget; Export-Optimierer als Strategien). Vollständig I/O-frei. Kein Import von Infrastruktur-Klassen.

↓ nur über Port-Interfaces

Port-Schicht

Technologieunabhängige Verträge. Die Domain formuliert, was sie braucht (z.

B. „schreibe GPX,“), aber nicht wie. Dies ermöglicht Mocking im Test ohne reale Abhängigkeiten.

↓ implementiert durch

Infrastruktur / Adapter

Realisiert die Ports: GPX-Serialisierung nach XML, HTTP-Client für W3W, Dateisystem-Zugriff, Systemuhr, SLF4J-Logging.

2.1.3 Kommunikationsregeln

Von	Nach	Erlaubt?	Begründung
Host	API	Ja	Die API-Fassade ist der einzige definierte Einstiegspunkt.
Host	Domain	Nein	Direkter Zugriff würde die Kapselungswirkung der API-Fassade aushebeln.
API	Domain	Ja	Orchestrierung der Fachlogik ist Kernaufgabe der API-Schicht.
API	Infrastruktur	Nein	Keine technischen Details im Application Service; nur Port-Interfaces.
Domain	Infrastruktur	Nein	Fachlogik bleibt I/O-frei; Infrastruktur wird über Ports entkoppelt.
Infrastruktur	Domain	Nein	Zirkuläre Abhängigkeit würde die Testbarkeit zerstören.

Tabelle 3: Erlaubte und verbotene Abhängigkeiten zwischen den Subsystemen.

2.1.4 Physische Sicht

Die Komponente wird als Bibliothek in den Host-Prozess eingebettet und läuft in dessen JVM. Für Integrationstests sind lokale Mocks der Ports ausreichend. Die Offline-Fähigkeit sichert den Betrieb auch ohne Netzwerkzugang.

2.2 Funktionen

Die folgende Tabelle fasst die Hauptfunktionsgruppen der Bibliothek zusammen. Die genaue Spezifikation der einzelnen Anforderungen folgt in Kapitel 3.

Funktionsgruppe	Beschreibung	Anforderungen
Session-Lebenszyklus	Start, laufende Aufzeichnung und Abbruch einer Peilungs-Session mit UUID-Identifikation.	/LF010/-/ LF090/
Peilung und Kurs	Berechnung von geografischem Azimut, Entfernung (Haversine) und diskreter Himmelsrichtung; optionale Kursabweichung.	/LF030/-/ LF050/
GPS-Track-Aufzeichnung	Kontinuierlicher Rohspeicher validierter GPS-Fixes; Segmentierung bei Zeitlücken; zweistufiges Punktbudget.	/LF100/-/ LF180/
GPX-Export	GPX-1.1-konformer Export des Tracks als String oder byte[]; optionales atomares Dateischreiben.	/LF200/-/ LF230/
Track-Optimierung	Austauschbare Strategie-Algorithmen: n-ter Punkt, Mindestabstand, Geraden-Heuristik, Douglas-Peucker.	/LF240/-/ LF270/
What3Words-Integration	Optionaler Reverse-Lookup von Koordinaten mit konfiguriertem Cache und TTL.	/LF280/, / LF290/
Validierung und Sicherheit	Koordinaten- und Zeitstempelprüfung; Path-Traversal-Schutz; XML-Escaping.	/LF300/-/ LF350/
Betrieb und Qualität	Deterministische Testbarkeit via Clock-Injection; strukturiertes Logging; Build-Reproduzierbarkeit.	/LF340/-/ LF500/

Tabelle 4: Hauptfunktionsgruppen der Java-Peilungskomponente mit Verweis auf Anforderungen.

Weitere Entwurfsprinzipien:

Prinzip	Entwurfsentscheidung	Messbarer Nutzen
Hohe Kohäsion	Module sind strikt nach fachlicher Verantwortung getrennt.	Anpassungen am GPX-Format bleiben lokal auf den <code>bearing-adapter-gpx</code> begrenzt.
Schwache Koppelung	Komponenten kommunizieren ausschließlich über APIs statt über konkrete Implementierungen.	Unterkomponenten lassen sich flexibler austauschen, was Mocking für Unit-Tests vereinfacht.
Information Hiding	Domänen-Logik wird hinter einer Fassade gekapselt. Daten fließen als unveränderliche Value Objects.	Die interne Struktur ist geschützt; externe Aufrufer können den Systemzustand nicht schädigen.
Separation of Concerns	Saubere Trennung zwischen Kernlogik und Infrastruktur (Dateizugriff, Netzwerk-I/O).	Fachlogik lässt sich ohne Infrastruktur-Overhead testen und umgekehrt.
Wiederverwendbarkeit	Einsatz des Strategy-Patterns für Policies und Optimizer.	Neue Algorithmen lassen sich ohne Eingriffe in den bestehenden Kontrollfluss integrieren.

Tabelle 5: Entwurfsprinzipien und deren messbarer Nutzen.

2.3 Benutzerprofile

Als Bibliothek ohne eigene Benutzeroberfläche kennt die Peilungskomponente keine direkte Endnutzer-Interaktion. Stattdessen werden Stakeholder und Benutzerrollen im Sinne der IEEE-830-Norm definiert.

2.3.1 Stakeholder und Einflussmatrix

Rolle	Erwartung / Interesse	Einfluss
Dozent / Prüfer	Nachweis der Vorlesungsinhalte, lauffähiger Code, formal saubere Dokumentation nach IEEE-/SOPHIST-Standard.	hoch
Studierenden-team	Wartbare, erweiterbare Architektur; klare Testbarkeit; nachvollziehbare Anforderungen.	mittel
Host-Entwickler/in	Stabile, vollständig dokumentierte API; klare Fehlersemantik über maschinenlesbare Exception-Codes.	hoch
Endnutzer/in (indirekt)	Zuverlässige Peilungswerte und korrekte GPX-Ausgabe in der Host-Anwendung.	mittel
Betrieb	Strukturiertes Logging auf WARN-Level; keine stillen Fehler; deterministisches Verhalten.	niedrig-mittel

Tabelle 6: Stakeholder der Java-Peilungskomponente mit Erwartungen und Einfluss.

2.3.2 Primärer Akteur: Host-Anwendung

Der primäre Akteur ist die **Host-Anwendung**. Sie ruft die öffentliche Java-API der Bibliothek auf und übernimmt folgende Aufgaben:

- Bereitstellung von GNSS-Fixes (lat, lon, Zeitstempel, optional HDOP, Geschwindigkeit, Elevation).
- Steuerung des Session-Lebenszyklus (Start, Update, Complete/Abort).
- Entscheidung über Speicherort und Dateiverwaltung beim GPX-Export.
- Verwaltung des What3Words-API-Keys (sofern genutzt).
- Registrierung von Listnern für Ereignisbenachrichtigungen.

2.3.3 Sekundärer Akteur: Externe Dienste

Als optionaler sekundärer Akteur fungiert der **What3Words-Dienst** (W3W): Er beantwortet HTTPS-Reverse-Lookup-Anfragen, die die Bibliothek über ihren konfigurierten HTTP-Adapter stellt. Die Bibliothek greift nie direkt auf GNSS-Hardware zu.

2.3.4 Funktionale Anforderungen – Übersicht

Die normativen Detail-Spezifikationen mit Aktivitätsdiagrammen stehen in **Kapitel 3.1**. Die folgende Tabelle fasst die Anforderungen auf Produktebene zusammen.

/LF / /LL	Name	Kurzbeschreibung	Verwandte Anforderungen
/LF010/	Session starten	Ziel setzen, Konfiguration validieren, Zustand ACTIVE.	/LF400/
/LF030/	Positionsupdate	Validieren, Track aktualisieren, Snapshot ableitbar.	/LF100/
/LF050/	Peilungswerte lesen	Azimet, Distanz, Ordinalrichtung, optionaler Kursfehler.	–
/LF060/	Regulär beenden	Zustand COMPLETED, GPX erzeugen.	/LF200/
/LF070/	Abbrechen	Zustand ABORTED, GPX-Daten ohne Pflichtdatei.	/LF200/, /LF230/
/LF200/	Optimieren + Export	Strategiewahl, Serialisierung, optionales Dateis Schreiben.	/LF240/–/LF270/, /LF490/
/LF280/	W3W auflösen	Optionaler Reverse-Lookup mit Cache.	/LF290/
/LF280/	W3W-Key fehlt	Fallback-String, kein Netzwerkaufruf.	Variante Kap. 3.1
/LF280/	Netzwerk-Timeout W3W	WARN-Log, Fallback ohne Crash.	Variante Kap. 3.1
/LF230/	GPX als Bytes	byte[] UTF-8 ohne BOM.	/LF060/, /LF070/
/LF080/	Listener wirft	Exception gefangen, Session bleibt konsistent.	/LF010/–/LF070/
/LL160/	Reset nach Ende	IDLE erreichbar nach COMPLETED/ABORTED.	/LF010/

Tabelle 7: Übersicht funktionaler Anforderungen der Java-Peilungskomponente (Traceability zu Kap. 3.1).

2.4 Einschränkungen

Kapitel 1.2 hat den Funktionsumfang bereits umrissen. Die folgende Auflistung hält ihn als verbindliche Abgrenzung fest und ergänzt die Aspekte Validierung und Qualitätssicherung.

2.4.1 Im Scope

- Berechnung von Zielrichtung (Azimut), Entfernung und Himmelsrichtung (Ordinalrichtung) auf WGS84-Basis, bei verfügbarem Kurswert auch der Kursabweichung.
- Klar definierte Integrationsschnittstellen, über die der Host Positionsdaten und zugehörige Metadaten (z. B. Zeitstempel, Geschwindigkeit, Genauigkeit) bereitstellt.
- Verwaltung des Session-Lebenszyklus aus Start, laufender Aufzeichnung und Abbruch. Nach einem Abbruch bleiben die bis dahin erfassten Track-Daten für den Export erhalten.
- Konfigurierbare Aufzeichnung: Punktbudget (Soft-/Hard-Limit), Segmentierung bei Zeitlücken und Validierung der Eingaben (Koordinaten, Zeit). Wie oft Positionsupdates eintreffen, steuert der Host.
- Export der Track-Daten als GPX 1.1, wahlweise als String oder Bytefolge.
- Wählbare Optimierungsverfahren vor dem Export (n-ter Punkt, Mindestabstand, Geraden-Heuristik, Douglas-Peucker) sowie eine optionale What3Words-Auflösung mit lokalem Caching.
- Qualitätssicherung durch automatisierte Unit-Tests der Kernfunktionen — insbesondere der geografischen Berechnungen, der Optimierungsalgorithmen und der GPX-Serialisierung.

2.4.2 Außerhalb des Scopes

- Optimierung der eingehenden Track-Daten während der laufenden Aufzeichnung.
- Magnetische Peilung sowie die automatische Korrektur der magnetischen Deklination.
- Kartendarstellung, Map-Matching, Routing und Geocoding allgemeiner Adressen (ausgenommen die genannte What3Words-Integration).
- Direkte Hardwareanbindung (z. B. GNSS-Treiber, Sensorfusion); die rohen Messwerte liefert ausschließlich der Host.
- Feste Persistenzvorgaben wie ein vorgegebenes Dateiziel für den GPX-Export; über Speicherort und Dateiverwaltung entscheidet der Host.
- Cloud-Persistenz, Benutzer- und Rechteverwaltung.

2.5 Annahmen und Abhängigkeiten

Dieses Kapitel beschreibt die Annahmen und externen Abhängigkeiten, auf denen die definierten Anforderungen basieren. Änderungen an diesen Faktoren können Auswirkungen auf Anforderungen, Schnittstellen und das Systemverhalten haben.

2.5.1 Annahmen

ID	Annahme	Einfluss auf Anforderungen und Verhalten	Folge bei Verletzung
A1	Der Host liefert fertige Positions- und Kursdaten über die öffentliche API; die Bibliothek liest keine Sensoren und führt keine Sensorfusion aus.	Systemgrenze und Funktionsumfang (Peilung und Track aus bereitgestellten Messwerten) bleiben definiert.	Ohne valide Host-Daten sind Peilung und Aufzeichnung nicht sinnvoll nutzbar; dies liegt außerhalb der Verantwortung der Bibliothek.
A2	Nur der Host steuert die Aufruffrequenz; die Bibliothek dünnt den eingehenden Datenstrom nicht von sich aus aus.	Roh-Trackfüllung, Punktbudget und Segmentierung folgen der vom Host gewählten Updatefrequenz.	Andere Taktungsmodelle würden die dokumentierten Erwartungen verletzen und Anpassungen am SRS nach sich ziehen.
A3	Geografische Berechnungen und Eingaben beziehen sich auf WGS84-konforme Koordinaten, wie in der Spezifikation vorausgesetzt.	Azimut, Distanz und Exportformate bleiben konsistent zur fachlichen Definition.	Abweichende Bezugssysteme erfordern eine Überarbeitung der Anforderungen und Algorithmen.
A4	Die Host-JVM unterstützt mindestens Java 11.	Bytecode, APIs und Tooling der Bibliothek sind darauf ausgelegt.	Ältere JVMs werden nicht unterstützt; ein Wechsel der Mindestversion wäre eine SRS-Änderung.
A5	Integratoren und Tests können eine Clock injizieren, um zeitabhängiges Verhalten deterministisch abzusichern.	Reproduzierbare Tests und nachvollziehbare Session-Zeitlogik.	Ohne geeignete Zeitquelle können zeitbasierte Regeln in Tests schwer stabil gehalten werden.

Tabelle 8: Übersicht der getroffenen Systemannahmen

2.5.2 Abhängigkeiten

Kategorie	Abhängigkeit	Details und Konsequenz
-----------	--------------	------------------------

Host	Konfiguration (W3W)	API-Schlüssel und Adapter-Konfiguration bei Nutzung von What3Words. Ohne Schlüssel oder Netz gelten dokumentierte Fallbacks; volle W3W-Funktionalität entfällt.
Host	Dateisystem (GPX)	Zielpfade, ggf. <code>SafeFileSink</code> und Schreibrechte. Bei Verletzung schlägt GPX-Persistenz fehl oder wird abgelehnt; Fehler über definierte Mechanismen sichtbar.
Host	Logging	SLF4J-API der Bibliothek; konkretes Backend bindet der Host. Ohne Binding keine oder unerwartete Logausgabe; Fachlogik unberührt.
Host	Netzwerk	HTTPS zur W3W-API nur bei aktivierter Nutzung; Erreichbarkeit und TLS-Vertrauen im Host-Umfeld. Eingeschränkter oder ausgefallener Reverse-Lookup und Cache-Update.
Laufzeit	Java SE	Version 11 oder höher; Ausführungsumgebung im Host-Prozess.
Build	Apache Maven	Projekt unter <code>implementation/</code> ; Abhängigkeiten über Maven Central. Kompilieren, Testen, Packen; Änderungen am Build können das SRS ergänzen lassen.
Laufzeit	<code>slf4j-api</code>	2.0.12 (Parent-POM). Fassade für Logging; Versionwechsel kann Integrationshinweise betreffen.
Test	JUnit 5, AssertJ, Mockito, Jimps	Versionen 5.10.2, 3.25.3, 5.11.0, 1.3.0. Nur für automatisierte Tests; nicht Teil der Host-Laufzeit.
Extern	What3Words-API	Optional über HTTPS. Profil <code>w3w</code> steuert optionale Integrationstests im Build. Verfügbarkeit und API-Änderungen liegen außerhalb der Bibliothek.
Architektur	Eingebetteter Betrieb	Kein eigener Serverprozess der Bibliothek; keine Abhängigkeit von einem separaten, von der Komponente bereitgestellten Dienst.

Tabelle 9: Übersicht der Systemabhängigkeiten und externen Voraussetzungen

3 Spezifische Anforderungen

Dieses Kapitel bildet den normativen Kern des Dokuments. Die funktionalen Anforderungen sind nach SOPHIST/IEEE-830 formuliert: jede /LF.../- bzw. /LL.../-Spezifikation enthält Allgemeines, Funktionsbedingungen und Funktionsablauf mit zugehörigem Aktivitätsdiagramm. Ergänzt wird das Kapitel durch nicht-funktionale Anforderungen (3.2), externe Schnittstellen (3.3), Performance-Anforderungen (3.4) und das Qualitätsmodell nach ISO/IEC 25010 (3.5).

3.1 Funktionale Anforderungen

3.1.1 Akteure

Primär- und Sekundärakteure der Java-Peilungskomponente:

Akteur	Beschreibung	Beispiel
Host-App	Beliebige Java-Anwendung, die die Bibliothek einbettet. Steuert den Session-Lebenszyklus und übergibt GNSS-Fixes.	Desktop-App, REST-Backend, Embedded-System
Team (Entwicklung)	Das Entwicklungsteam konfiguriert und baut die Bibliothek; relevant für Build- und Wartungsanforderungen.	Studierenden-team DHBW Stuttgart

Tabelle 10: Primärakteure der Java-Peilungskomponente.

Sekundärakteure unterstützen das System oder werden vom System genutzt:

Akteur	Beschreibung
What3Words-API	Externer HTTPS-Dienst für Koordinaten-Reverse-Lookup. Optional; Timeout und Fallback abgesichert.
Dateisystem	Optionaler Ausgabekanal für GPX-Dateien. Zugriff nur bei expliziter Host-Konfiguration.
SLF4J-Backend	Empfängt strukturierte Log-Events der Bibliothek; konkrete Implementierung ist Host-Sache.

Tabelle 11: Sekundärakteure der Java-Peilungskomponente.

/ LF010/	Allgemein			
	Funktion	Session starten		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF400/, /LF080/, IEEE 830, Projektauftrag SWE
	Include	—	Datenzugriffe	/LD100/, /LD110/
	Beschreibung	Die Host-App startet eine neue Peilungs-Session. Das System prüft Zielkoordinate und Konfiguration, vergibt eine UUID, friert die Konfiguration ein und wechselt in den Zustand ACTIVE. Alle registrierten Listener werden benachrichtigt.		
	Funktionsbedingungen			
	Auslöser	Der Nutzer der Host-App startet eine neue Navigations- oder Peilungssitzung. Die Host-App ruft daraufhin <code>startSession(target, config)</code> auf.		
	Vorbedingung	1. Die Bibliothek befindet sich im Zustand <code>IDLE</code>. 2. <code>GeoCoordinate target</code> ist nicht null und enthält valide WGS84-Werte. 3. <code>SessionConfig config</code> ist nicht null; alle Pflichtfelder sind positiv belegt.		
	Nachbedingungen (Erfolg)	• <code>sessionId</code> (UUID) wurde dem Host zurückgegeben. • Zustand = <code>ACTIVE</code>; <code>startedAt</code> gesetzt; <code>SessionConfig</code> eingefroren. • Alle Listener haben <code>onSessionStarted</code> empfangen (oder Fehler protokolliert).		
	Nachbedingungen (Fehler)	Aktive Session vorhanden: <code>BearingStateException(ALREADY_ACTIVE)</code> — /LF020/. Konfiguration ungültig: <code>ValidationException(INVALID_CONFIG)</code> . Zielkoordinate ungültig: <code>ValidationException(COORD_RANGE)</code> .		
	Funktionsablauf			
	Standardablauf	1. Host übergibt <code>GeoCoordinate target</code> und <code>SessionConfig config</code>. 2. System prüft: keine Session im Zustand <code>ACTIVE</code>. 3. System validiert <code>SessionConfig</code> und <code>target</code> (WGS84). 4. System erzeugt UUID (RFC 4122 v4) und setzt <code>startedAt</code>. 5. System friert <code>SessionConfig</code> ein und wechselt zu <code>ACTIVE</code>. 6. System feuert <code>onSessionStarted(sessionId)</code> an alle Listener. 7. System gibt <code>sessionId</code> synchron zurück.		
	Alternativablauf	2a – Aktive Session: <code>BearingStateException(ALREADY_ACTIVE)</code> ; kein Zustandswechsel. 3a – Config ungültig: <code>ValidationException(INVALID_CONFIG)</code> . 3b – Koordinate ungültig: <code>ValidationException(COORD_RANGE)</code> .		

		6a – Listener wirft: WARN-Log; restliche Listener weiter benachrichtigt.
	Sonstiges	Systemgrenzen: Kein GNSS-Hardwarezugriff; keine eigenen Threads; kein Netzwerk. , Spezielle Anforderungen: UUID RFC 4122 v4; ClockPort- Injection; immutable SessionConfig. , Hinweise: Ein reset() ist nötig, um erneut zu starten (/LL160/).

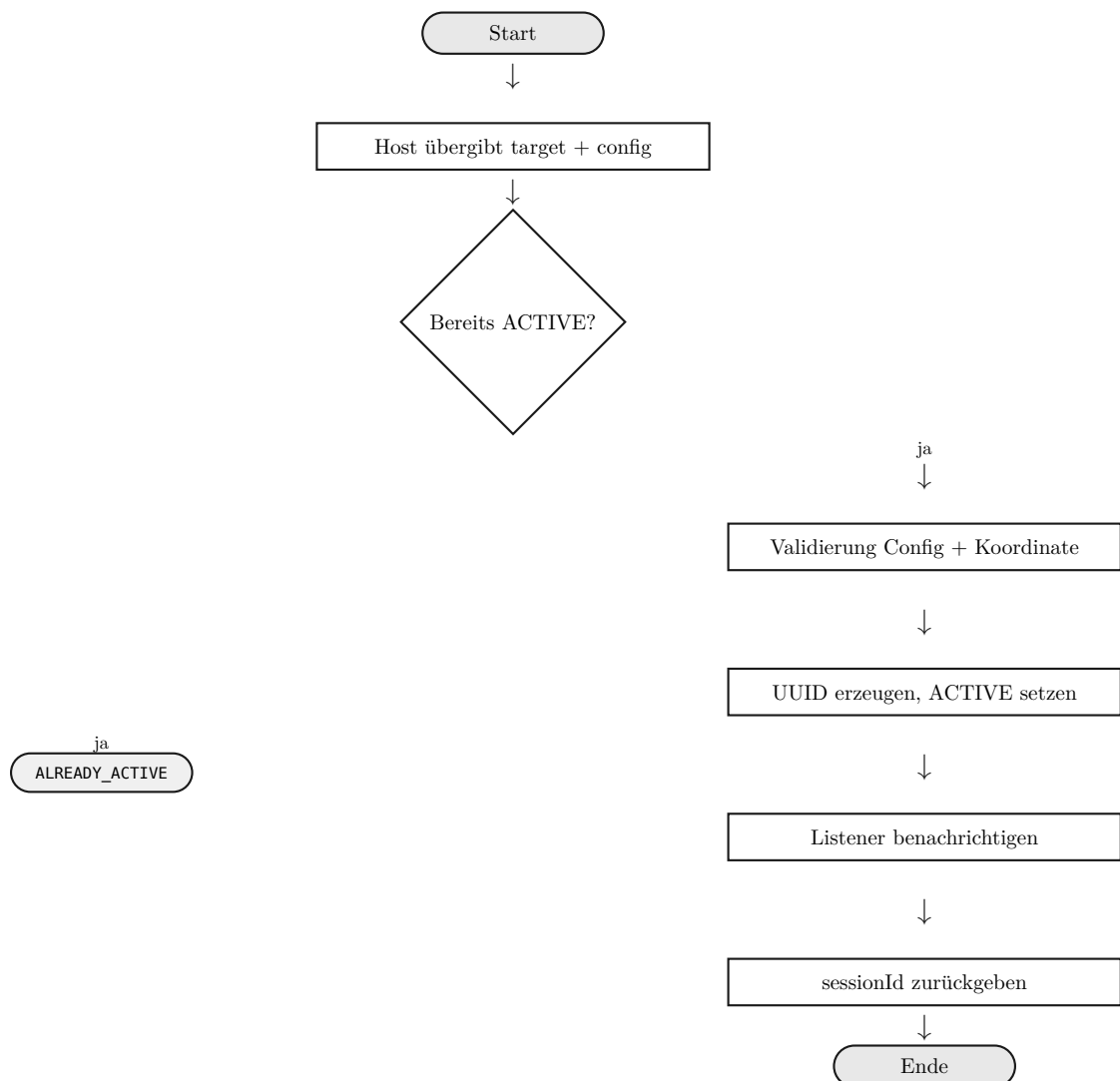


Abbildung 1: Aktivitätsdiagramm /LF010/

/ LF030/	Allgemein			
	Funktion	Positionsupdate verarbeiten		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF010/, /LF100/, /LF050/, Projektauftrag SWE
	Include	/LF100/ Track-Speicherung	Datenzugriffe	/LD120/, /LD130/, /LD140/
	Beschreibung	Die Host-App liefert einen neuen GPS-Fix. Das System validiert Koordinaten und Zeitstempel, prüft Segment- und Budgetgrenzen, speichert den Fix im Rohspeicher, berechnet Azimut und Distanz und benachrichtigt alle Listener.		
	Funktionsbedingungen			
	Auslöser	GNSS-Stack liefert Positionsbestimmung; Host ruft <code>onPositionUpdate(GpsFix fix)</code> auf.		
	Vorbedingung	1. Session im Zustand <code>ACTIVE</code> (/LF010/). 2. <code>GpsFix</code> ist nicht null. 3. Hard-Limit noch nicht erreicht (sonst Alternativfall 5a).		
	Nachbedingungen (Erfolg)	Fix im Rohspeicher; Azimut, Distanz und Ordinalrichtung aktuell; Listener benachrichtigt. Bei Limit: entsprechendes Limit-Event gefeuert.		
	Nachbedingungen (Fehler)	Koordinaten ungültig: <code>ValidationException(COORD_RANGE)</code> . Zeitstempel ungültig: <code>ValidationException(TIMESTAMP_INVALID)</code> . Hard-Limit STOP: kein weiterer Punkt gespeichert.		
	Funktionsablauf			
	Standardablauf	1. Host übergibt <code>GpsFix fix</code> . 2. System validiert lat/lon (WGS84) und Zeitstempel. 3. System prüft Segmentlücke und Punktbudget (Soft/Hard-Limit). 4. System speichert Fix im Rohspeicher (/LF100/). 5. System berechnet Azimut, Distanz, Ordinal; optional Kursabweichung. 6. System feuert <code>onPositionProcessed(BearingSnapshot)</code> an alle Listener.		
	Alternativablauf	2a – Koordinaten ungültig: Exception; kein Punkt gespeichert. 2b – Zeitstempel ungültig: Exception; kein Punkt gespeichert. 3b – Erster Fix: eröffnet Segment 1 automatisch. 5a – Hard-Limit STOP: <code>onHardLimitReached</code> ; Session bleibt <code>ACTIVE</code> . 6a – Listener wirft: Exception gefangen; restliche Listener weiter.		

	Sonstiges	Systemgrenzen: Kein Sensorzugriff; nicht thread-sicher; Host steuert Aufrufhäufigkeit. , Spezielle Anforderungen: Azimut $\pm 1^\circ$ für $D > 10$ m; Segmentierung konfigurierbar; /LL020/. , Hinweise: Optionale Felder ohne Normierung; negative hdop/speed $\rightarrow$ <code>ValidationException(INVALID_FIELD_VALUE)</code>.
--	-----------	--

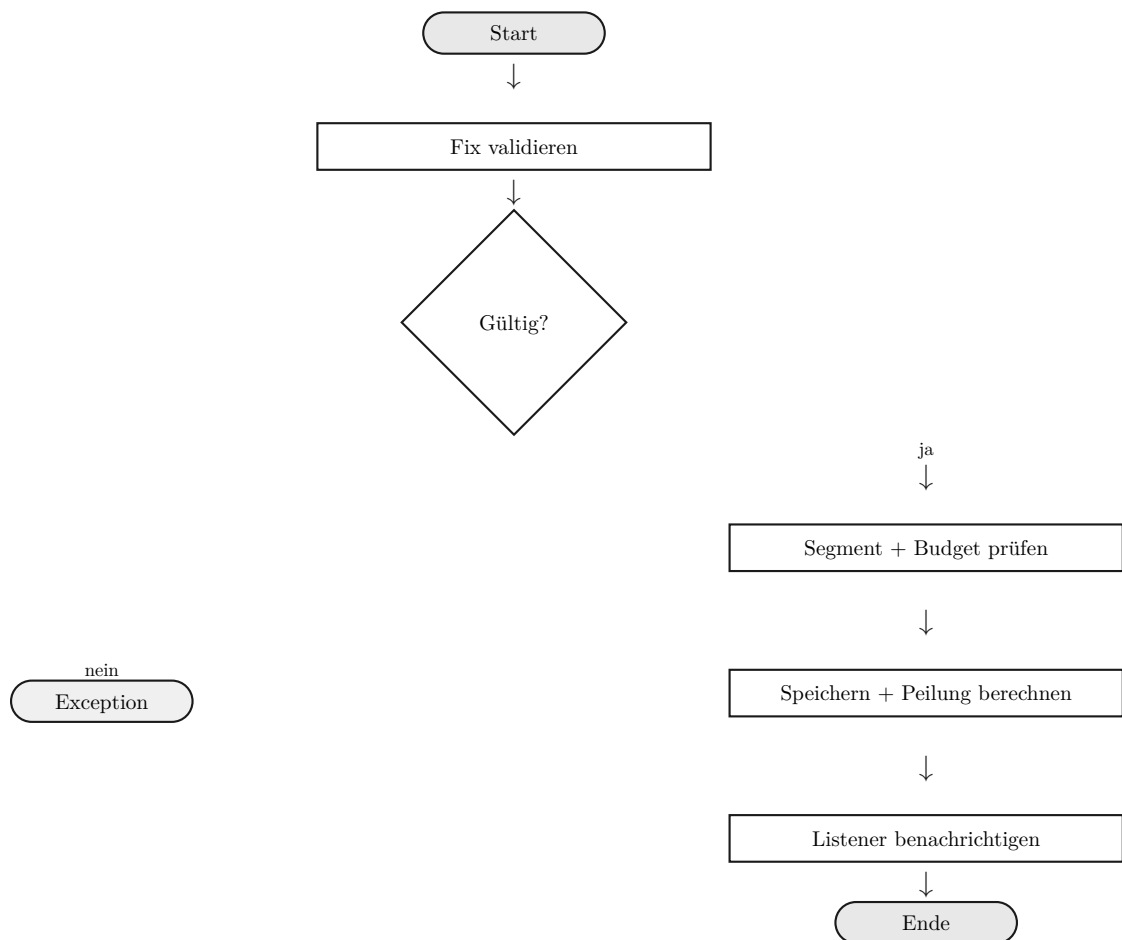


Abbildung 2: Aktivitätsdiagramm /LF030/

/ LF050/	Allgemein			
	Funktion	Peilungswerte lesen		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF030/, Projektauftrag SWE
	Include	—	Datenzugriffe	/LD140/
	Beschreibung	Die Host-App fragt den aktuellen Peilungsschnappschuss ab. Das System gibt Azimut, Entfernung zum Ziel und diskrete Himmelsrichtung zurück. Optional Kursabweichung, falls Kurs bekannt.		
	Funktionsbedingungen			
	Auslöser	Host ruft <code>getSnapshot()</code> zur UI-Aktualisierung auf.		
	Vorbedingung	1. Session im Zustand <code>ACTIVE</code> . 2. Mindestens ein gültiger Fix wurde über /LF030/ verarbeitet (für befüllten Snapshot).		
	Nachbedingungen (Erfolg)	<code>BearingSnapshot</code> mit Azimut, Distanz, Ordinal; optional <code>bearingErrorDeg</code> . Kein interner Zustand verändert.		
	Nachbedingungen (Fehler)	Session nicht <code>ACTIVE</code> oder kein Fix: <code>Optional.empty()</code> — kein Fehler, kein Log.		
	Funktionsablauf			
	Standardablauf	1. Host ruft <code>getSnapshot()</code> auf. 2. System prüft: mindestens ein Fix im Rohspeicher? 3. System leitet <code>compassOrdinal</code> aus <code>azimuthDeg</code> ab (8-teilig). 4. System befüllt optionale Kursabweichung. 5. System gibt <code>ImmutableOptional<BearingSnapshot></code> zurück.		
	Alternativablauf	2a – Kein Fix: <code>Optional.empty()</code> . 1a – Nicht <code>ACTIVE</code>: <code>Optional.empty()</code> (Javadoc).		
	Sonstiges	Systemgrenzen: Nur lesend; kein Netzwerk/Dateizugriff. , Spezielle Anforderungen: <code>Immutable Value Object</code> ; 45°-Segmentierung; nie null-Rückgabe. , Hinweise: Snapshot = Zustand nach letztem Fix; kein Live-Tracking zwischen Aufrufen.		

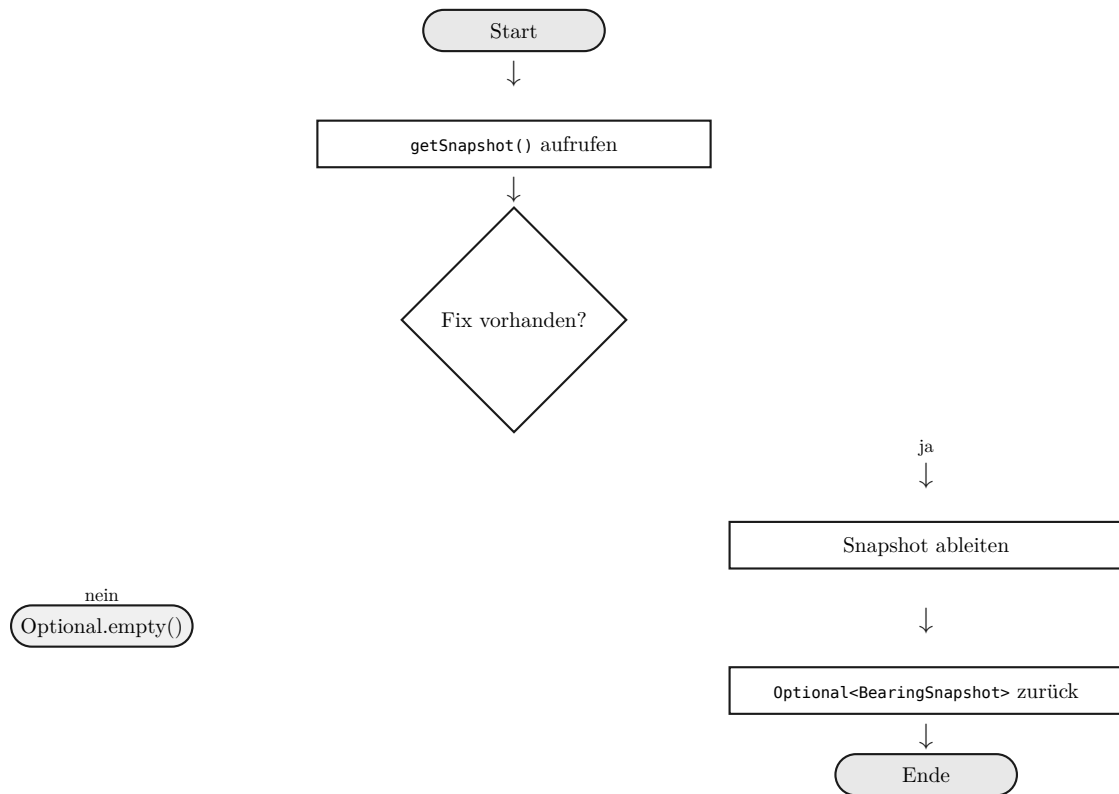


Abbildung 3: Aktivitätsdiagramm /LF050/

/ LF060/	Allgemein			
	Funktion	Peilung regulär beenden		
	Akteur	Host-App (Primär). Sekundär: Dateisystem (optional).	Verweise	/LF010/, /LF200/, Projektauftrag SWE
	Include	/LF200/ GPX-Export	Datenzugriffe	/LD150/
	Beschreibung	Die Host-App schließt eine laufende Peilungs-Session regulär ab. Das System wechselt zu COMPLETED, erzeugt GPX 1.1, berechnet Statistiken und benachrichtigt Listener.		
	Funktionsbedingungen			
	Auslöser	Navigationsaufgabe abgeschlossen; Host ruft complete() auf.		
	Vorbedingung	1. Session im Zustand ACTIVE. 2. Kein gleichzeitiger abort()-Aufruf.		
	Nachbedingungen (Erfolg)	Zustand = COMPLETED; GpxResult und SessionStats zurückgegeben; Listener benachrichtigt; optional GPX-Datei geschrieben.		
	Nachbedingungen (Fehler)	Session nicht ACTIVE: BearingStateException(SESSION_ENDED). Dateischieben fehlgeschlagen: BearingIOException; Zustand dennoch COMPLETED.		

Funktionsablauf	
Standardab- lauf	1. Host ruft <code>complete()</code> auf. 2. System setzt <code>endedAt</code> und Zustand <code>COMPLETED</code>. 3. System erstellt immutable View des Rohtracks. 4. System führt GPX-Export aus (<code>/LF200/</code>). 5. System berechnet <code>SessionStats</code> und feuert <code>onSessionCompleted</code>. 6. System gibt <code>GpxResult</code> zurück.
Alternativ- ablauf	1a – Nicht ACTIVE: <code>SESSION_ENDED</code>. 4a – IO-Fehler: <code>BearingIOException</code>; <code>GpxResult</code> gültig. 5a – Listener wirft: WARN-Log; restliche Listener weiter.
Sonstiges	Systemgrenzen: Optimierung nur auf Export-Kopie; Path-Tra- versal-Schutz. , Spezielle Anforderungen: GPX 1.1 Namespace; ISO-8601 UTC; atomares Schreiben; Export < 2 s / 10k Punkte. , Hinweise: Nach <code>complete()</code> ist <code>reset()</code> für neue Session nötig (<code>/LL160/</code>).

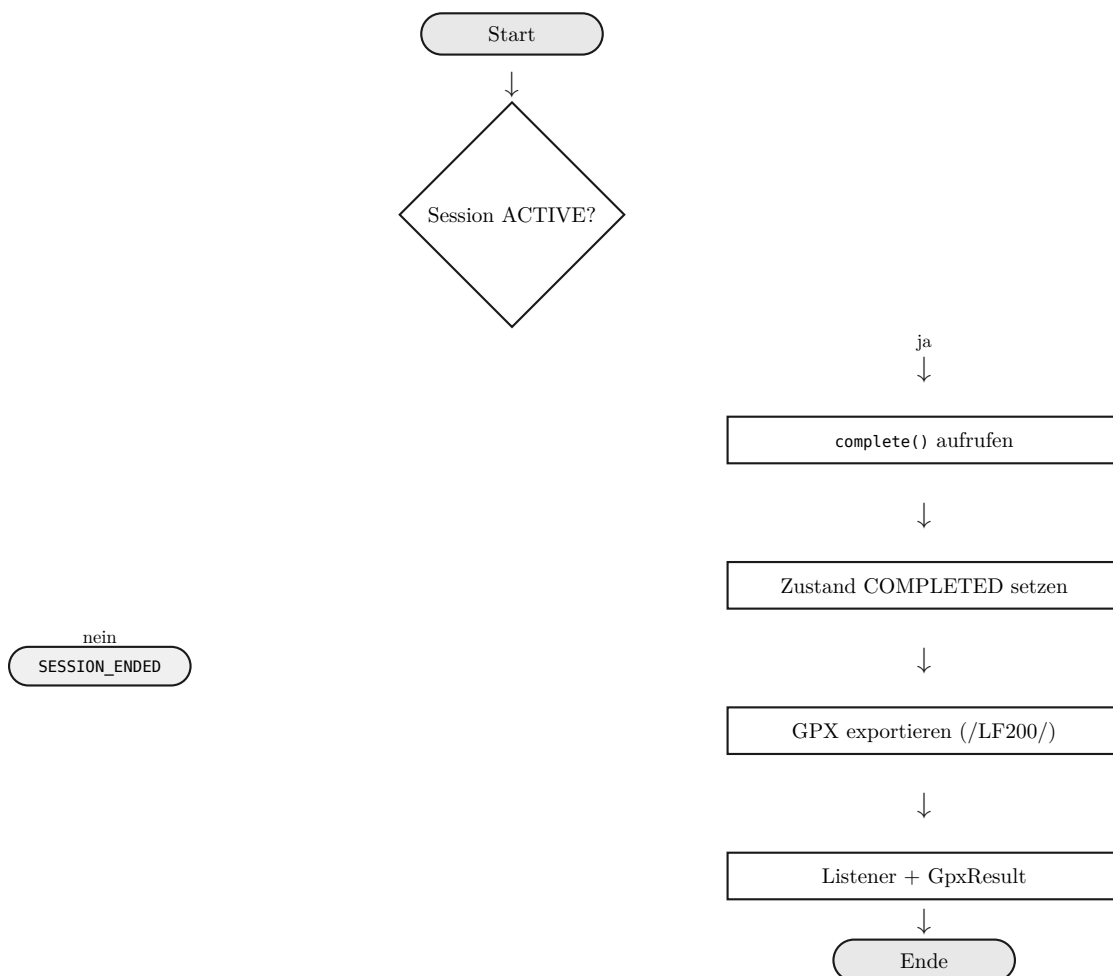


Abbildung 4: Aktivitätsdiagramm /LF060/

/ LF070/	Allgemein			
	Funktion	Peilung abbrechen		
	Akteur	Host-App (Primär). Sekundär: Dateisystem (optional).	Verweise	/LF010/, /LF200/, /LF230/, Klärungsge-spräch Peter Bohl
	Include	/LF200/ GPX-Export	Datenzu-griffe	/LD150/
	Beschrei-bung	Die Host-App bricht eine laufende Session vorzeitig ab. Track-Daten bleiben als GPX abrufbar. Datei nur bei <code>persistOnAbort = true</code> .		
	Funktionsbedingungen			
	Auslöser	Feldabbruch, Verbindungsabbruch oder externe Steuerlogik; Host ruft <code>abort()</code> auf.		
	Vorbedin-gung	1. Session im Zustand <code>ACTIVE</code> . 2. Kein gleichzeitiger <code>complete()</code> -Aufruf.		
	Nachbedin-gungen (Er-folg)	Zustand = <code>ABORTED</code> ; <code>GpxResult</code> mit allen Punkten bis Abort; <code>SessionStats</code> berechnet.		
	Nachbedin-gungen (Feh-ler)	Session nicht <code>ACTIVE</code> : <code>BearingStateException(SESSION_ENDED)</code> .		
	Funktionsablauf			
	Standardab-lauf	1. Host ruft <code>abort()</code> auf. 2. System setzt <code>endedAt</code> und Zustand <code>ABORTED</code> . 3. System serialisiert GPX 1.1 (/LF200/). 4. Optional: atomares Dateisichreiben bei <code>persistOnAbort = true</code> . 5. System feuert <code>onSessionAborted</code> und gibt <code>GpxResult</code> zurück.		
	Alternativ-ablauf	1a – Nicht <code>ACTIVE</code>: <code>SESSION_ENDED</code> . 3a – Leerer Track: valides GPX mit leerem <code><trk></code> . 4a – <code>persistOnAbort false</code>: kein Dateizugriff. 4b – IO-Fehler: <code>BearingIOException</code> ; <code>GpxResult</code> gültig.		
	Sonstiges	Systemgrenzen: GPX-Export keine Pflicht; Pfad obliegt Host. , Spezielle Anforderungen: Track-Daten werden nicht verworfen; leerer Track = valides GPX. , Hinweise: Primäre Methode für Ausnahmefälle.		

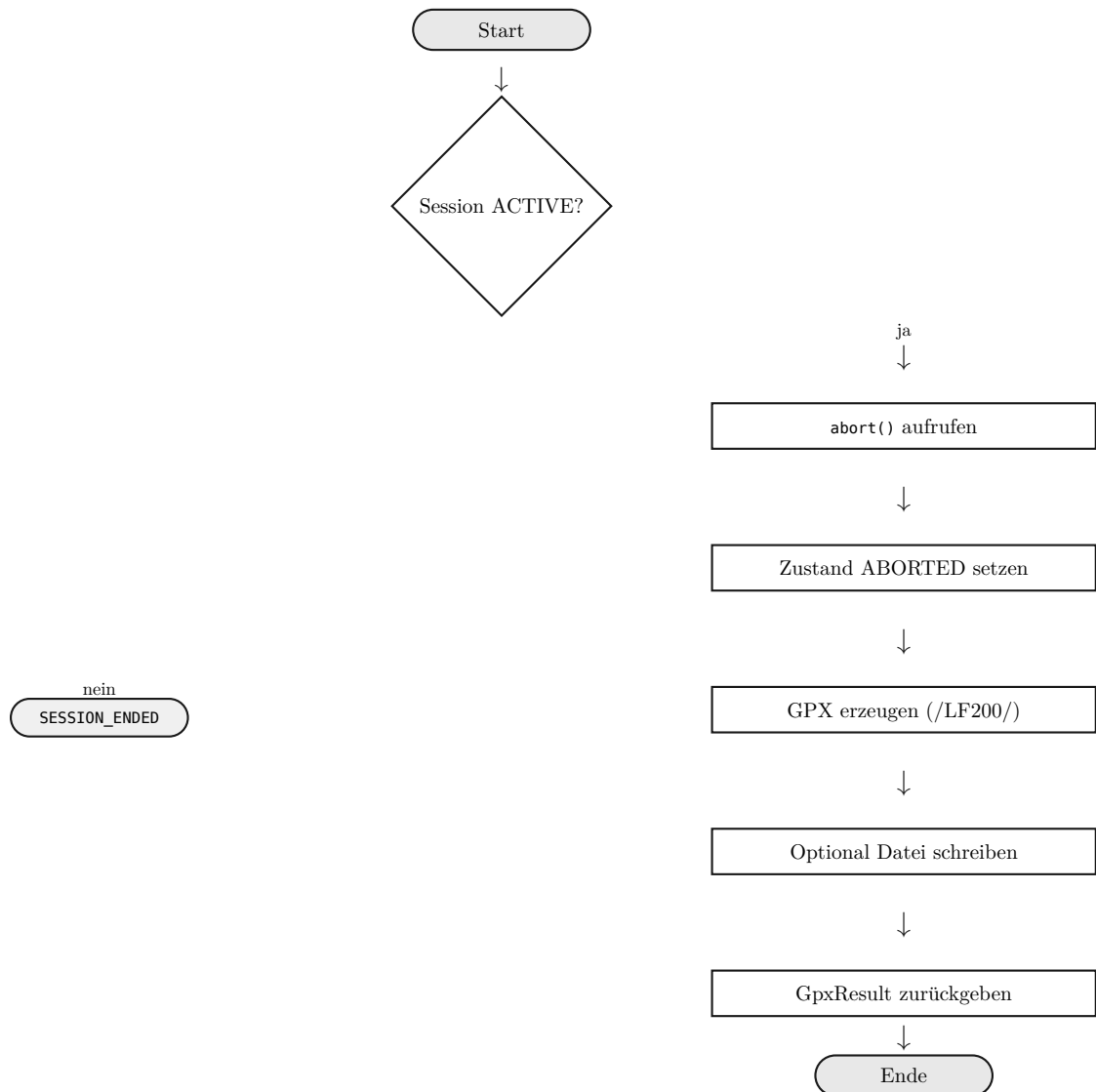


Abbildung 5: Aktivitätsdiagramm /LF070/

/ LF200/	Allgemein			
	Funktion	GPX exportieren und optimieren		
	Akteur	Host-App (Primär). Sekundär: Dateisystem (optional).	Verweise	/LF060/, /LF070/, GPX 1.1, OWASP Path Traversal
	Include	/LF240/-/LF270/ Optimierer, /LF490/	Datenzugriffe	/LD120/, /LD150/
	Beschreibung	Nach Abschluss oder Abbruch erzeugt das System ein GPX-1.1-Dokument. Optional Optimierungspipeline. Export als String, Byte-Array und optional Datei.		
	Funktionsbedingungen			
	Auslöser	Intern durch /LF060/ (complete()) oder /LF070/ (abort()) ausgelöst.		
	Vorbedingung	1. Session COMPLETED oder ABORTED. 2. Rohtrack als immutable View verfügbar (auch leer).		
	Nachbedingungen (Erfolg)	Valides GpxResult (String + Bytes); optional Datei auf Disk; Rohspeicher unverändert.		
	Nachbedingungen (Fehler)	Path-Traversal: SecurityException(PATH_TRAVERSAL). Dateisystemfehler: BearingIOException; GpxResult dennoch gültig.		
	Funktionsablauf			
	Standardablauf	1. System klonet Rohtrack. 2. System wendet registrierte TrackOptimizer an (/LF240/-/LF270/). 3. System erstellt GPX-Metadaten und serialisiert XML 1.1. 4. System escaped Strings und kodiert UTF-8 ohne BOM. 5. Optional: Pfad prüfen und atomar schreiben. 6. System gibt GpxResult zurück.		
	Alternativablauf	2a – Keine Optimizer: Rohpunkte direkt exportieren. 5a – Path-Traversal: SecurityException; kein Schreiben. Leerer Track: GPX mit leerem <trk>.		
	Sonstiges	Systemgrenzen: Kein Netzwerk; Douglas-Peucker bis 50 km Segment. , Spezielle Anforderungen: Namespace Pflicht; /LL130/ XML-Escaping; Export < 2 s / 10k Punkte. , Hinweise: Optimierung wirkt nur auf Export-Kopie.		

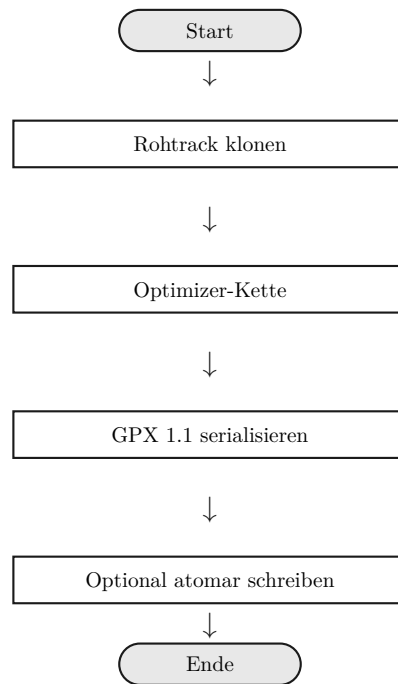


Abbildung 6: Aktivitätsdiagramm /LF200/

/ LF280/	Allgemein			
	Funktion	What3Words-Adresse auflösen		
	Akteur	Host-App (Primär). Sekundär: W3W-API, LRU-Cache.	Verweise	/LF290/, What3Words API Docs
	Include	—	Datenzu- griffe	/LD160/
	Beschrei- bung	Host löst WGS84-Koordinate in W3W-Dreiwortkombination auf. Cache-Lookup zuerst; bei Miss HTTPS-Anfrage. Bei Fehler Fall- back-String.		
	Funktionsbedingungen			
	Auslöser	Host ruft resolveW3w(GeoCoordinate coordinate) auf.		
	Vorbedin- gung	1. w3wApiKey konfiguriert und nicht leer (sonst Variante ohne Key). 2. Koordinate valide (WGS84).		
	Nachbedin- gungen (Er- folg)	W3W-String zurückgegeben; bei Erfolg Cache-Eintrag mit TTL.		
	Nachbedin- gungen (Feh- ler)	Kein Key, Timeout, HTTP-Fehler: Fallback "w3w://unavailable" + WARN-Log. Koordinate ungültig: ValidationException(COORD_RANGE).		
	Funktionsablauf			
	Standardab- lauf	1. Host ruft resolveW3w(coordinate) auf. 2. System normalisiert Koordinate (Cache-Schlüssel). 3. Cache-Treffer mit gültiger TTL? → Rückgabe. 4. HTTPS-Request an W3W-API (Timeout 5 s). 5. JSON parsen, Cache aktualisieren, words zurückgeben.		
	Alternativ- ablauf	1a – Kein API-Key: Fallback sofort (siehe /LF280/ Variante „Ohne Key“). 4a – Timeout: Fallback + WARN-Log. 4b – HTTP 4xx/5xx: Fallback + WARN-Log. 2a – Koordinate ungültig: ValidationException.		
	Sonstiges	Systemgrenzen: Opportunistisch; Fehler beeinträchtigt Session nicht. , Spezielle Anforderungen: LRU-Cache 256 Einträge, TTL 1 h; Maven-Profil w3w. , Hinweise: Max. 1 Retry konfigurierbar.		

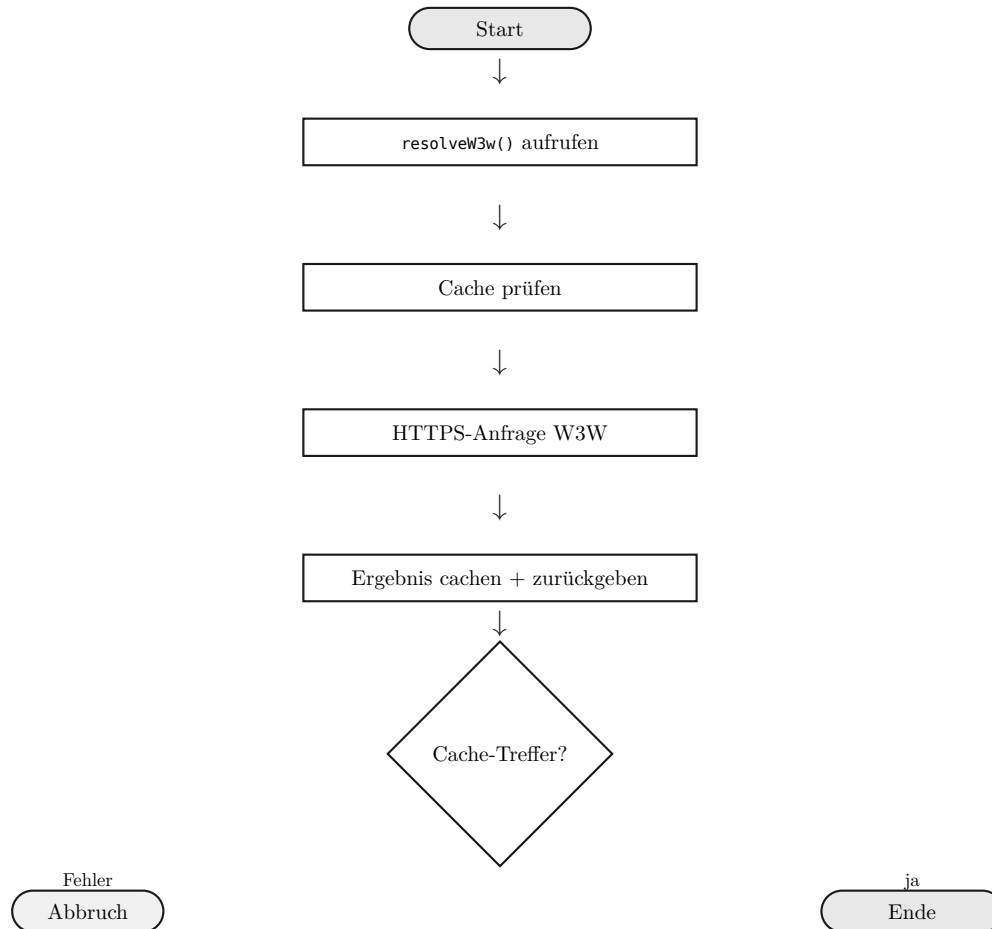


Abbildung 7: Aktivitätsdiagramm /LF280/

/ LF280/	Allgemein			
	Funktion	W3W-Fallback ohne API-Key		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF280/ Lookup-Hauptfunktion
	Include	—	Datenzugriffe	—
	Beschreibung	Host ruft W3W-Lookup ohne API-Key auf. System verzichtet auf Netzwerk und gibt Fallback-String zurück.		
	Funktionsbedingungen			
	Auslöser	w3wApiKey leer oder nicht gesetzt bei resolveW3w().		
	Vorbedingung	w3wApiKey in SessionConfig ist leer oder Optional.empty().		
	Nachbedingungen (Erfolg)	Rückgabe "w3w://unavailable"; WARN-Log; kein Netzwerkaufruf.		

	Nachbedin- gungen (Feh- ler)	–
	Funktionsablauf	
	Standardab- lauf	1. System stellt fest: kein API-Key. 2. WARN-Log mit <code>sessionId</code> , Ursache <code>NO_API_KEY</code> . 3. Rückgabe <code>"w3w://unavailable"</code> .
	Alternativ- ablauf	–
	Sonstiges	Systemgrenzen: Kein Netzwerkaufruf ohne API-Key. , Spezielle Anforderungen: MDC-Feld <code>sessionId</code> im WARN-Log. , Hinweise: In Javadoc von <code>resolveW3w()</code> dokumentiert.

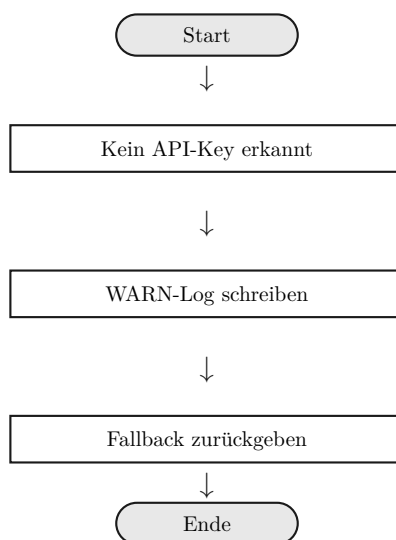


Abbildung 8: Aktivitätsdiagramm /LF280/ (ohne Key)

/ LF280/	Allgemein			
	Funktion	W3W-Netzwerk-Timeout		
	Akteur	Host-App (Primär). Sekundär: W3W-API (nicht erreichbar).	Verweise	/LF280/ Look-up-Hauptfunktion, OWASP API Security
	Include	—	Datenzu- griffe	–
	Beschrei- bung	W3W-API antwortet nicht innerhalb des Timeout-Limits. System bricht ab, loggt WARN und gibt Fallback zurück. Session unbeeinträchtigt.		

Funktionsbedingungen	
Auslöser	HTTP-Client erkennt Timeout bei W3W-REST-Anfrage.
Vorbedingung	API-Key vorhanden; Netzwerkaufruf ausgelöst; Timeout überschritten.
Nachbedingungen (Erfolg)	Fallback zurückgegeben; WARN-Log; Session aktiv.
Nachbedingungen (Fehler)	–
Funktionsablauf	
Standardablauf	1. HTTP-Client erkennt Timeout. 2. System fängt Timeout-Exception intern. 3. WARN-Log (Dauer, URL, sessionId). 4. Rückgabe "w3w://unavailable".
Alternativablauf	–
Sonstiges	Systemgrenzen: Timeout konfigurierbar, Standard 5 s. , Spezielle Anforderungen: Exception darf nicht propagieren; kein unkontrollierter Thread. , Hinweise: CI-Test mit Mock-Server empfohlen.

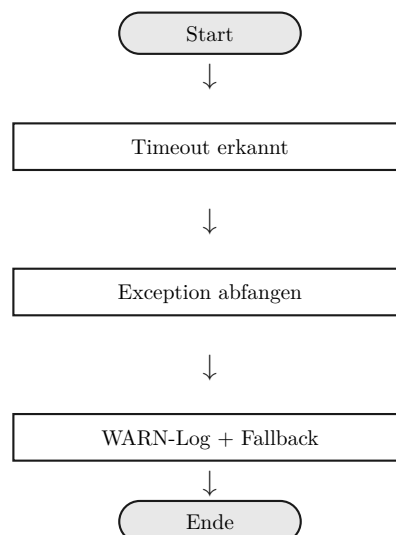


Abbildung 9: Aktivitätsdiagramm /LF280/ (Timeout)

/ LF230/	Allgemein			
	Funktion	GPX als Byte-Array exportieren		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF060/, /LF070/, /LF200/, RFC 3629
	Include	—	Datenzugriffe	/LD150/
	Beschreibung	Host greift auf GPX-Export als byte[] (UTF-8, kein BOM) zu — z. B. HTTP, OutputStream, Datenbank.		
	Funktionsbedingungen			
	Auslöser	Host benötigt GPX als Byte-Array nach Session-Ende.		
	Vorbedingung	Session COMPLETED oder ABORTED; GpxResult von /LF060/ oder /LF070/ vorhanden.		
	Nachbedingungen (Erfolg)	byte[] UTF-8 ohne BOM; Länge > 0 bei nicht-leerem Track.		
	Nachbedingungen (Fehler)	—		
	Funktionsablauf			
	Standardablauf	1. Host ruft GpxResult.gpxBytes() auf. 2. System gibt vorberechnetes byte[] zurück (kein Re-Serialisieren).		
	Alternativablauf	—		
	Sonstiges	Systemgrenzen: Kein Datei-/Netzwerkzugriff. , Spezielle Anforderungen: Byte-äquivalent zu gpxString; kein BOM; immutable Cache. , Hinweise: Leerer Track liefert minimales valides GPX-Byte-Array.		

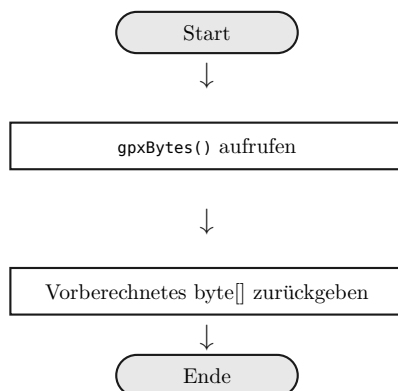


Abbildung 10: Aktivitätsdiagramm /LF230/

/LF080/	Allgemein			
	Funktion	Listener-Exception isolieren		
	Akteur	Host-App (Primär; fehlerhafte Listener-Implementierung).	Verweise	/LF010/, /LF030/, /LF060/, /LF070/, Observer-Pattern
	Include	—	Datenzugriffe	—
	Beschreibung	Ein <code>BearingListener</code> wirft bei Callback eine <code>RuntimeException</code> . System fängt ab, loggt und benachrichtigt restliche Listener. Session bleibt konsistent.		
	Funktionsbedingungen			
	Auslöser	Fehlerhafter Listener bei Event-Aufruf (/LF010/–/LF070/).		
	Vorbedingung	Listener registriert; wirft <code>RuntimeException</code> bei Callback.		
	Nachbedingungen (Erfolg)	Restliche Listener benachrichtigt; WARN-Log; Session konsistent.		
	Nachbedingungen (Fehler)	—		
	Funktionsablauf			
	Standardablauf	1. System ruft <code>listener.onXxx(payload)</code> auf. 2. Listener wirft <code>RuntimeException</code> . 3. System fängt Exception intern. 4. WARN-Log mit Typ, Message, <code>sessionId</code> (MDC). 5. Iteration mit nächstem Listener fortsetzen.		
	Alternativablauf	—		

	Sonstiges	Systemgrenzen: Gilt für alle <code>BearingListener</code>-Callbacks. , Spezielle Anforderungen: Contract in Javadoc; keine Abbruch der gesamten Schleife. , Hinweise: Postel's Law — tolerant gegenüber fehlerhaftem Host-Code.
--	-----------	---

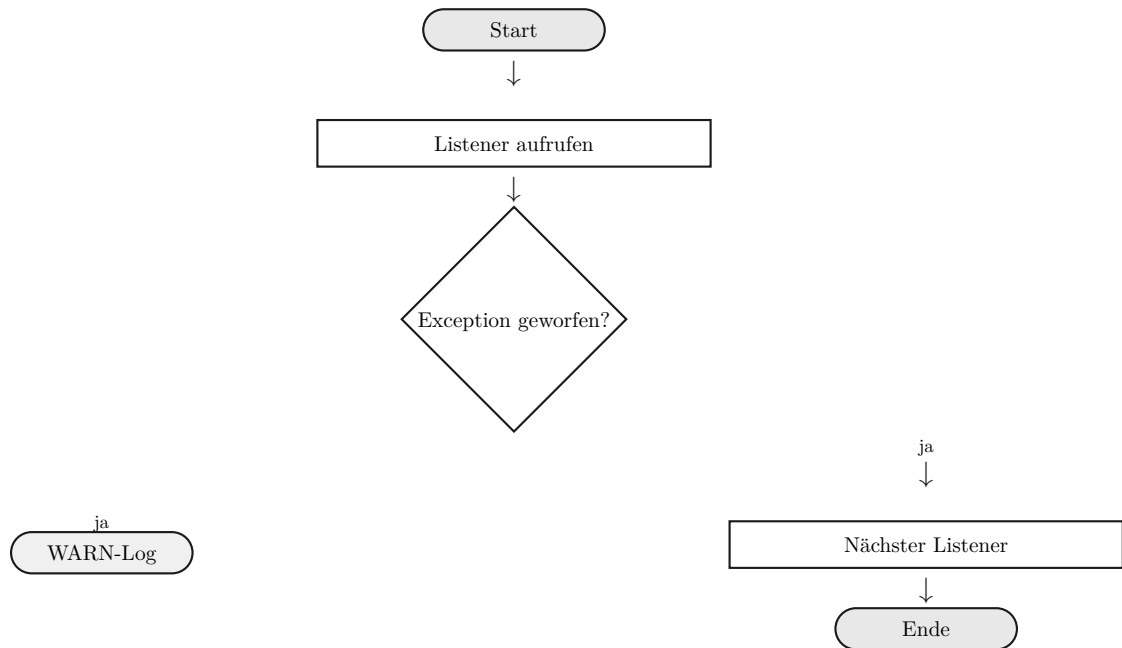


Abbildung 11: Aktivitätsdiagramm /LF080/

/ LL160/	Allgemein			
	Funktion	Session zurücksetzen		
	Akteur	Host-App (Primär). Sekundär: keine.	Verweise	/LF010/, /LF060/, /LF070/
	Include	—	Datenzugriffe	—
	Beschreibung	Host setzt Bibliothek nach COMPLETED/ABORTED auf IDLE zurück. Neue Session (/LF010/) ohne JVM-Neustart möglich.		
	Funktionsbedingungen			
	Auslöser	Host möchte nach /LF060/ oder /LF070/ erneut starten; ruft reset() auf.		
	Vorbedingung	Session im Zustand COMPLETED oder ABORTED.		
	Nachbedingungen (Erfolg)	Zustand = IDLE; interne Felder gelöscht; startSession() wieder aufrufbar.		
	Nachbedingungen (Fehler)	Session ACTIVE: BearingStateException(SESSION_STILL_ACTIVE); kein Reset.		
	Funktionsablauf			
	Standardablauf	1. Host ruft reset() auf. 2. System prüft Zustand COMPLETED oder ABORTED. 3. System löscht Session-Felder (UUID, Zeiten, Rohspeicher, Peilung). 4. System wechselt zu IDLE. 5. /LF010/ kann erneut aufgerufen werden.		
	Alternativablauf	2a – Session ACTIVE: SESSION_STILL_ACTIVE; kein Reset.		
	Sonstiges	Systemgrenzen: Kein Netzwerk/Dateizugriff beim Reset. , Spezielle Anforderungen: W3W-LRU-Cache wird nicht geleert (sessions-übergreifend). , Hinweise: Voraussetzung für Langzeit-Betrieb ohne JVM-Neustart.		

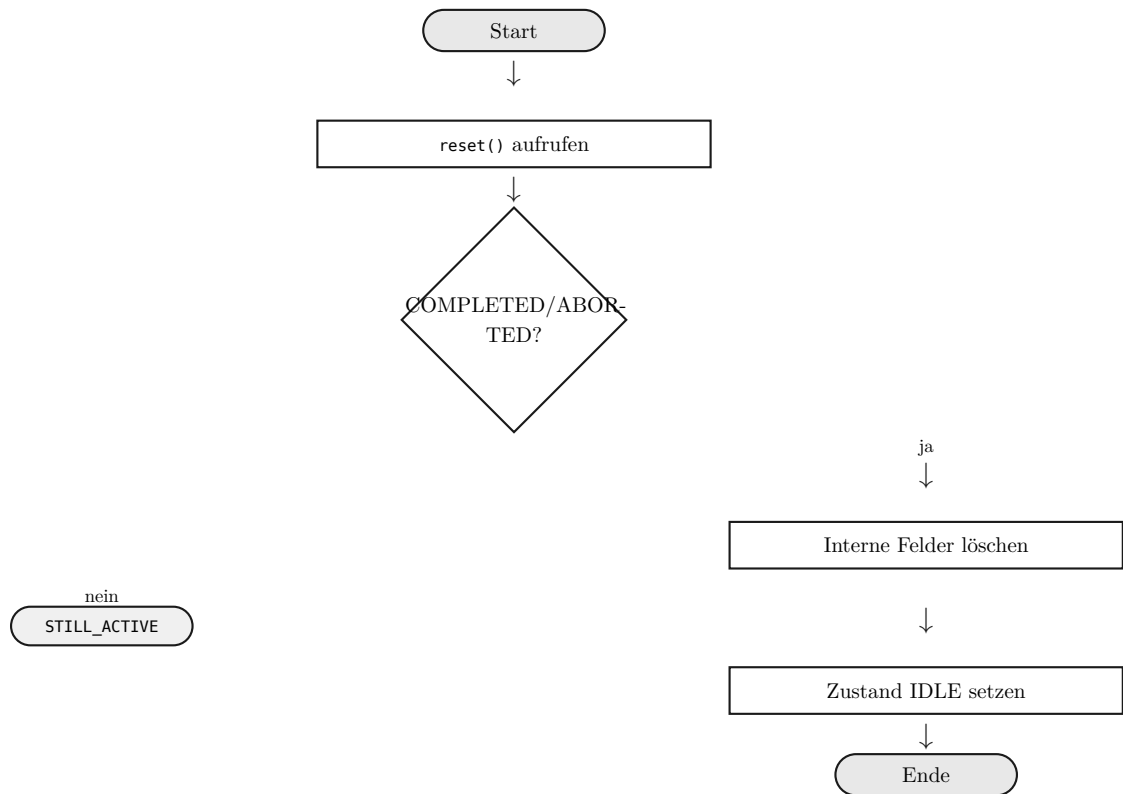


Abbildung 12: Aktivitätsdiagramm /LL160/

3.2 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben **wie** das System seine Aufgaben erfüllt. Sie sind durch /LL.../-Bezeichner referenzierbar und als messbare Qualitätsmerkmale formuliert.

/ LL010/	Funktion	Eingabevalidierung		
	Quelle	Qualitätsrichtlinie	Verweise	/LF030/, /LF010/
	Beschreibung	Ungültige Eingaben (null, WGS84-Bereich verletzt, Zeitstempel außerhalb Toleranz) dürfen keine undefinierten Zustände erzeugen. Alle Fehlerpfade müssen durch dedizierte Testfälle abgesichert sein; 100 % der Exception-Pfade in Validierungsklassen müssen durch Tests erreichbar sein.		

/ LL020/	Funktion	Genauigkeit der Peilungsberechnung		
	Quelle	Fachanforderung	Verweise	/LF030/, /LF050/
	Beschreibung	Für Distanzen > 10 m darf der berechnete Azimut maximal $\pm 1^\circ$ von einer unabhängigen Referenz (Haversine-Formel) abweichen. Überprüfung für mindestens drei Referenzszenarien: Äquator, Polnähe ($\varphi > 80^\circ$), Stuttgart (48,77°N/9,18°E).		

/ LL030/	Funktion	Performance: Peilungszyklus		
	Quelle	Echtzeit-Anforderung	Verweise	/LF030/
	Beschreibung	Vollständiger Update-Zyklus (onPositionUpdate → Azimut/Distanz berechnen) soll auf Referenz-Laptop typisch < 1 ms, Worst-Case (ohne GC) < 100 ms betragen. Überschreitung muss im Test-Report dokumentiert werden.		

/ LL040/	Funktion	Performance: GPX-Export		
	Quelle	Performanceanforderung	Verweise	/LF200/
	Beschreibung	GPX-Export für 10.000 Trackpunkte ohne Optimierung soll auf Referenz-Laptop in unter 2 Sekunden erfolgen. Messung per JMH- oder @Timeout-Test in target/benchmark-report.txt.		

/ LL050/	Funktion	Speicherverbrauch		
	Quelle	Ressourcenanforderung	Verweise	/LF030/
	Beschreibung	Heap-Zuwachs der Bibliothek für 10.000 Punkte (alle optionalen Felder) soll typisch < 50 MB betragen. Validierung durch JVM-Heap-Snapshot-Test.		

/ LL100/	Funktion	Javadoc-Vollständigkeit		
	Quelle	Vorlesung SWE	Verweise	–
	Beschreibung	Gesamte öffentliche API (alle public-/protected-Klassen und Methoden) muss vollständig mit Javadoc dokumentiert sein. Checkstyle-Plugin im Maven-Build erzwingt dies; fehlende Javadoc führen zum Build-Fehler.		

/ LL110/	Funktion	Portabilität		
	Quelle	Projektauftrag	Verweise	–
	Beschreibung	Keine plattformspezifischen Pfade, Zeilentrennzeichen oder sun.*-APIs. Kompilierbar und ausführbar auf Windows 10+, macOS 12+ und Ubuntu 22.04 ohne Anpassungen. CI-Matrix auf mindestens zwei Plattformen empfohlen.		

/ LL120/	Funktion	Build-Reproduzierbarkeit		
	Quelle	Projektauftrag	Verweise	–
	Beschreibung	Maven-Build auf frisch geklontem Repository mit <code>mvn clean test</code> in unter 5 Minuten erfolgreich. Alle Abhängigkeiten versioniert und über Maven Central auflösbar. Keine SNAPSHOT-Abhängigkeiten im Release.		

/ LL130/	Funktion	Sicherheit: XML-Escaping		
	Quelle	Sicherheitsrichtlinie	Verweise	/LF200/
	Beschreibung	Alle Nutzerstrings in XML-Ausgaben müssen escaped werden (<code>&</code> , <code><</code> , <code>></code> , <code>"</code>). XXE-Schutz via <code>FEATURE_SECURE_PROCESSING = true</code> . Verifikation durch Integrationstest mit Sonderzeichen.		

/ LL140/	Funktion	Beobachtbarkeit via SLF4J		
	Quelle	Betriebsanforderung	Verweise	–
	Beschreibung	Alle WARN- und ERROR-Ereignisse via SLF4J protokolliert. Jeder Eintrag auf WARN+ enthält <code>sessionId</code> als MDC-Feld. MDC muss nach jedem Aufruf bereinigt werden (Thread-Pool-Schutz).		

/ LL150/	Funktion	Testabdeckung		
	Quelle	Vorlesung SWE	Verweise	/LL010/
	Beschreibung	Kernmodule <code>bearing-core</code> , <code>gps-tracker</code> , <code>gpx-exporter</code> : Zeilenabdeckung $\geq 85\%$ (JaCoCo). Kritische Klassen <code>BearingSession</code> ,		

		BearingCalculator: ≥ 90 %. Unterschreitung führt zum Build-Fehler (jacoco:check).
--	--	--

/ LL160/	Funktion	Session-Reset nach Abschluss		
	Quelle	Betriebsanforderung	Verweise	/LF010/, /LF060/, /LF070/
	Beschreibung	Nach <code>complete()</code> oder <code>abort()</code> und anschließendem <code>reset()</code> muss Zustand <code>IDLE</code> erreichbar sein, ohne JVM-Neustart. Verifikation durch Integrationstest: Session beenden → Reset → neuer Start mit neuer UUID.		

/ LL170/	Funktion	Determinismus der Tests		
	Quelle	Testbarkeitsanforderung	Verweise	–
	Beschreibung	ClockPort-Mocks mit fixer Zeit in allen zeitabhängigen Tests. <code>Thread.sleep()</code> und <code>System.currentTimeMillis()</code> im Testcode verboten. Testbaum ist idempotent (kein Abhängigkeit von Dateisystemresten).		

/ LL190/	Funktion	Determinismus bei gleichen Eingaben		
	Quelle	Qualitätsanforderung	Verweise	–
	Beschreibung	Identische Eingabesequenzen + ClockPort-Mock + selbe Konfiguration müssen stets byte-identischen GPX-String erzeugen. Verifikation durch Reproduzierbarkeitstests mit mindestens drei Durchläufen.		

/ LL200/	Funktion	Abhängigkeitslizenzierung		
	Quelle	Rechtliche Anforderung	Verweise	–
	Beschreibung	Standard-Build: nur permissiv lizenzierte Bibliotheken (Apache 2.0, MIT, BSD 2/3). W3W-Client nur in Maven-Profil <code>w3w</code> . Konformität via <code>license-maven-plugin:check</code> .		

3.3 Externe Schnittstellen

Die Bibliothek besitzt keine Benutzeroberfläche; ihre einzige Schnittstelle nach außen ist die öffentliche Java-API. Dieser Abschnitt beschreibt die aufrufbaren Methoden, die Hierarchie der geworfenen Ausnahmen und das Format der erzeugten GPX-Ausgabe. Zusammen bilden sie den Vertrag, auf den sich eine Host-Anwendung verlassen kann.

3.3.1 Öffentliche Java-API

Alle Aufrufe laufen über eine einzige Fassade. Sie spiegelt den Session-Lebenszyklus wider: Eine Session wird gestartet, mit Positions- und Kursdaten versorgt, abgefragt und schließlich regulär beendet oder abgebrochen. Jede Methode ist einer funktionalen Anforderung zugeordnet, was die Spalte *LF* festhält.

Signatur	Beschreibung	LF
<code>SessionId startSession(GeoCoordinate, SessionConfig)</code>	Neue Peilung starten; UUID zurückgeben.	/ LF010/
<code>void onPositionUpdate(GpsFix)</code>	GPS-Fix verarbeiten; Track + Peilung aktualisieren.	/ LF030/
<code>void onHeadingUpdate(double)</code>	Kurs aktualisieren (optional, 0–360°).	/ LF030/
<code>Optional<BearingSnapshot> getSnapshot()</code>	Aktuellen Schnappschuss abfragen.	/ LF050/
<code>GpxResult complete()</code>	Session regulär beenden; GPX erzeugen.	/ LF060/
<code>GpxResult abort()</code>	Session abbrechen; GPX erzeugen.	/ LF070/
<code>void reset()</code>	Auf IDLE zurücksetzen.	/ LL160/
<code>String resolveW3w(GeoCoordinate)</code>	W3W-Adresse auflösen (optional).	/ LF280/
<code>void addListener(BearingListener)</code>	Listener registrieren.	—
<code>void removeListener(BearingListener)</code>	Listener deregistrieren.	—

Tabelle 12: Vollständige öffentliche API-Methoden der Bibliothek.

3.3.2 Exception-Hierarchie

Fehler meldet die Bibliothek über ungeprüfte Ausnahmen (`RuntimeException`), damit die API frei von erzwungenen try-Blöcken bleibt. Alle fachlichen Fehler erben von der Basisklasse `BearingException` und tragen einen maschinenlesbaren `errorCode`, sodass eine Host-Anwendung gezielt auf einzelne Fälle reagieren kann, ohne Fehlertexte auszuwerten. Die `SecurityException` steht bewusst daneben: Sie signalisiert einen abgewehrten Path-Traversal-Versuch beim Dateischieben.

Exception-Hierarchie der Bibliothek

RuntimeException (JDK)

└─ SecurityException (*Path-Traversal bei Dateipersistenz; /LF200/*)

└─ BearingException (*Basisklasse; enthält errorCode: String, vollständig in Javadoc dokumentiert*)

└─ ValidationException (*Codes: COORD_RANGE · TIMESTAMP_INVALID · INVALID_CONFIG · INVALID_FIELD_VALUE*)

└─ BearingStateException (*Codes: ALREADY_ACTIVE · SESSION_ENDED · SESSION_STILL_ACTIVE*)

└─ BearingIOException (*Codes: IO_WRITE_ERROR*)

3.3.3 GPX 1.1-Ausgabeformat

Der Export folgt dem GPX-1.1-Schema von Topografix. Verbindlich sind der korrekte Namespace, UTF-8 ohne BOM sowie Zeitstempel als ISO-8601 in UTC (Suffix Z). Das folgende Beispiel zeigt den strukturellen Aufbau eines erzeugten Dokuments: Metadaten mit Namen, Startzeit und Bounding-Box, gefolgt von einem oder mehreren Track-Segmenten mit den einzelnen Trackpunkten.

GPX-1.1-Schemastruktur (normativ)

```
<?xml version="1.0" encoding="UTF-8"?>
<gpx version="1.1" creator="Java-Kompass/1.0"
  xmlns="http://www.topografix.com/GPX/1/1"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.topografix.com/GPX/1/1
    http://www.topografix.com/GPX/1/1/gpx.xsd">
  <metadata>
    <name>Peilung Stuttgart – Schlossplatz</name>
    <time>2026-06-12T10:00:00Z</time>
    <bounds minlat="48.775" minlon="9.182" maxlat="48.783" maxlon="9.191"/>
  </metadata>
  <trk>
    <trkseg>
      <trkpt lat="48.7758" lon="9.1829">
        <ele>248.0</ele>
        <time>2026-06-12T10:01:00Z</time>
        <hdop>1.2</hdop>
      </trkpt>
    </trkseg>
  </trk>
</gpx>
```

3.4 Anforderungen an die Performance

3.4.1 Quantitative Leistungskennwerte

Kennwert	Anforderung / Grenzwert	Messmethode
Peilungszyklus (Worst-Case)	< 100 ms ohne GC; typisch < 1 ms.	JMH / @Timeout-Test
GPX-Export (10k Punkte)	< 2 s ohne Optimierung.	JMH-Benchmark
Heap-Zuwachs (10k Punkte)	Typisch < 50 MB.	JVM-Heap-Snapshot
W3W Cache-Round-trip	< 1 ms bei Cache-Treffer.	Unit-Test mit Mock
Build (mvn clean test)	< 5 Minuten auf Referenz-CI.	CI-Laufzeit-Tracking

Tabelle 13: Quantitative Performance-Anforderungen mit Messmethode.

3.4.2 AD-01: Aktivitätsdiagramm onPositionUpdate

Dieses Diagramm zeigt den vollständigen Ablauf eines Positionsupdates (/LF030/). Es bildet die drei Entscheidungspunkte ab — Eingabevalidierung, Segmentgrenze und Punktbudget — sowie die anschließende Berechnung der Peilungsgrößen und die Benachrichtigung der Listener. Schlägt die Validierung fehl, bricht der Ablauf mit einer Ausnahme ab, ohne den Track zu verändern.

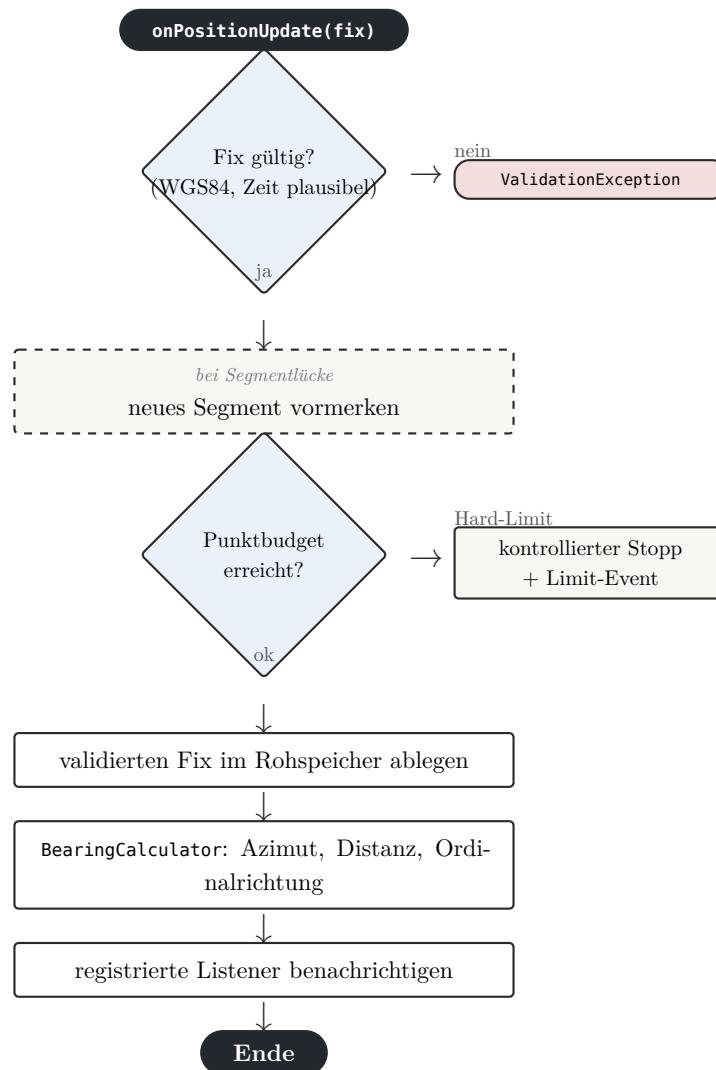


Abbildung 13: AD-01: Ablauf von onPositionUpdate (/LF030/) mit Validierungs-, Segment- und Budgetprüfung.

3.4.3 AD-02: Aktivitätsdiagramm GPX-Export

Dieses Diagramm zeigt den Exportvorgang (/LF200/). Der Rohtrack wird zunächst als unveränderlicher Snapshot festgehalten, damit der laufende Speicher unangetastet bleibt. Sind Optimierer konfiguriert, durchläuft der Track die Optimierer-Kette; andernfalls wandern die Rohpunkte unverändert weiter. Anschließend entstehen Metadaten und das fertige XML.

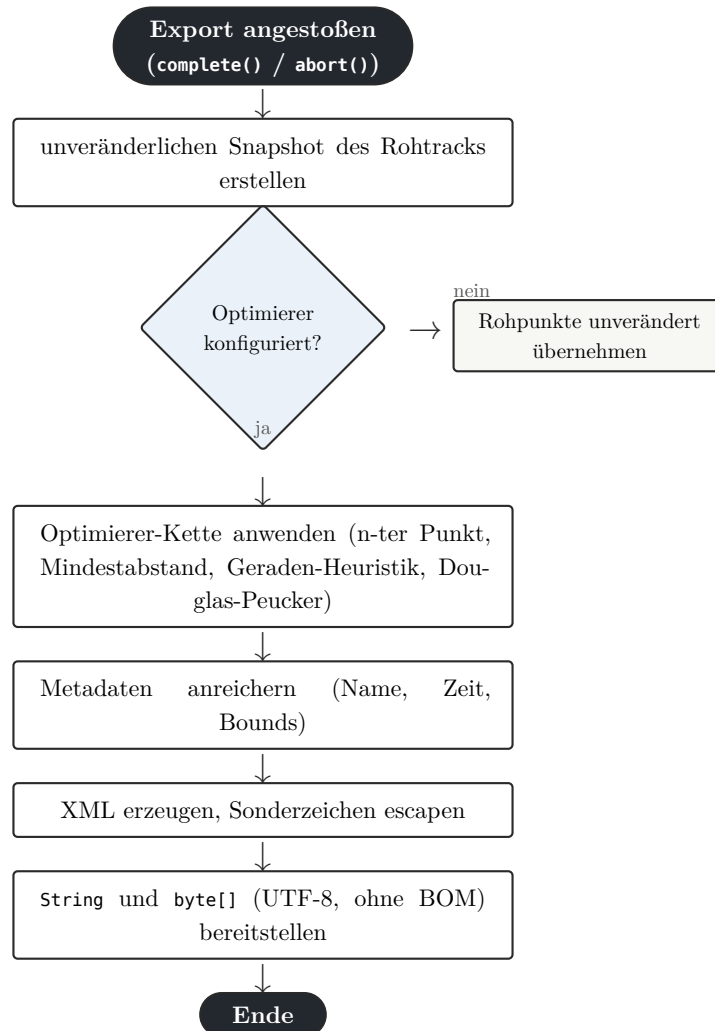


Abbildung 14: AD-02: Ablauf des GPX-Exports (/LF200/) mit optionaler Optimierer-Kette.

3.4.4 AD-03: Aktivitätsdiagramm Session starten

Dieses Diagramm zeigt den Start einer Session (/LF010/). Zwei Vorbedingungen werden geprüft: Es darf keine Session laufen, und Zielkoordinate sowie Konfiguration müssen gültig sein. Erst danach vergibt das System eine UUID, friert die Konfiguration ein und wechselt in den Zustand ACTIVE.

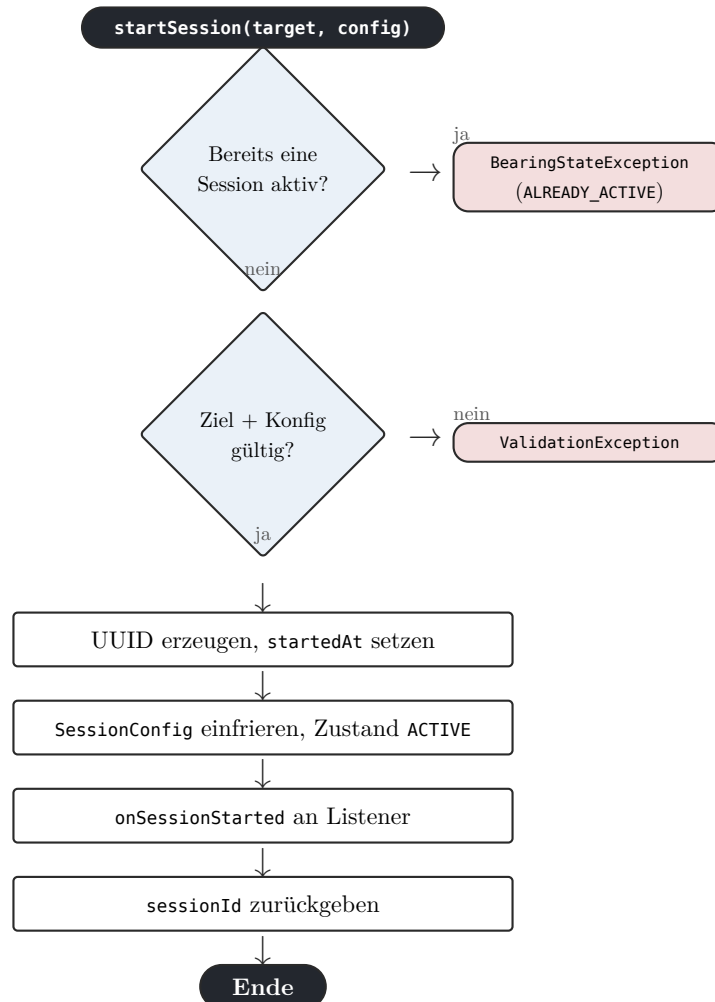


Abbildung 15: AD-03: Ablauf von `startSession (/LF010/)` mit Zustands- und Eingabeprüfung.

3.5 Qualitäts-Anforderungen

3.5.1 Qualitätsmodell nach ISO/IEC 25010

ISO/IEC 25010 beschreibt Produktqualität anhand von acht Hauptmerkmalen mit ihren Unterkriterien. Die folgende Matrix gewichtet jedes Unterkriterium nach seiner Bedeutung für genau dieses Projekt — von *sehr wichtig* bis *nicht relevant*. Die Gewichtung macht die Schwerpunkte sichtbar: Hoch eingestuft sind vor allem die Richtigkeit der Berechnung, die Zuverlässigkeit und die Wartbarkeit, während Aspekte wie Verbrauchsverhalten oder Anpassbarkeit für eine eingebettete Bibliothek dieser Größe nachrangig sind.

Produktqualität	sehr wichtig	wichtig	normal	nicht relevant
Funktionalität				
Angemessenheit	x			
Sicherheit			x	
Interoperabilität			x	
Konformität		x		
Ordnungsmäßigkeit			x	
Richtigkeit		x		
Zuverlässigkeit				
Fehlertoleranz			x	
Konformität			x	
Reife		x		
Wiederherstellbarkeit		x		
Benutzbarkeit				
Attraktivität	x			
Bedienbarkeit		x		
Erlernbarkeit	x			
Konformität			x	
Verständlichkeit		x		
Effizienz				
Konformität			x	
Zeitverhalten			x	
Verbrauchsverhalten				x
Änderbarkeit				
Analysierbarkeit			x	
Konformität			x	
Modifizierbarkeit		x		
Stabilität		x		
Testbarkeit			x	
Übertragbarkeit				

Anpassbarkeit				x
Austauschbarkeit		x		
Installierbarkeit			x	
Koexistenz		x		
Konformität			x	

Tabelle 14: Gewichtung der Qualitätsmerkmale nach ISO/IEC 25010 (Relevanzstufen je Unterkriterium).

3.5.2 Produktdaten (/LD.../)

Die Produktdaten beschreiben die zentralen Datenstrukturen der Bibliothek mit ihren Feldern, Typen und Wertebereichen. Sie bilden die Brücke zwischen den funktionalen Anforderungen und der späteren Implementierung: Jede /LD.../-Karte verweist auf die Anforderungen, in denen die Struktur entsteht oder verwendet wird.

/LD100/	Speicherinhalt	Peilungs-Session	
	Verweise	/LF010/, /LF060/, /LF070/	
	Attribute	sessionId	UUID RFC 4122 v4, 36 Zeichen
		status	Enum: IDLE · ACTIVE · COMPLETED · ABORTED
		target	GeoCoordinate (WGS84, validiert)
		startedAt	Instant UTC (gesetzt bei Start)
		endedAt	Optional<Instant> (gesetzt bei complete/abort)
		config	SessionConfig (immutable, eingefroren)

/LD110/	Speicherinhalt	Konfiguration (SessionConfig)	
	Verweise	/LF010/	
	Attribute	softLimitPoints	int ≥ 0 (0 = deaktiviert)
		hardLimitPoints	int ≥ softLimit (0 = deaktiviert)
		overflowMode	Enum: STOP · DOWNSAMPLE
		segmentGapThreshold	Duration > 0 (Standard: PT5M)

		maxFixAge	Duration > 0 (Standard: PT24H)
		persistOnAbort	boolean (Standard: false)
		w3wApiKey	Optional<String>
		w3wCacheTtl	Duration (Standard: PT1H)
		optimizers	List<TrackOptimizer>

/ LD120/	Speicherinhalt	GPS-Track	
	Verweise	/LF030/, /LF200/	
	Attribute	segments	List<TrackSegment> (mind. 1)
		totalPoints	int (abgeleitete Kennzahl)
		bounds	Optional<GeoBounds> (min/max lat/lon)

/ LD130/	Speicherinhalt	GPS-Punkt (GpsPoint)	
	Verweise	/LF030/	
	Attribute	lat	double $\in [-90,0, +90,0]$
		lon	double $\in [-180,0, +180,0]$
		time	Instant UTC (Pflichtfeld)
		ele	Optional<Double> (Höhe in m)
		hdop	Optional<Double> (≥ 0)
		speed	Optional<Double> (≥ 0 m/s)

/ LD140/	Speicherinhalt	Peilungs-Snapshot	
	Verweise	/LF050/	
	Attribute	azimuthDeg	double $\in [0,0^\circ, 360,0^\circ]$, [distanceM], [double ≥ 0], [compassOrdinal], [Enum: N · NE · E · SE · S · SW · W · NW], [bearingErrorDeg], [Optional<Double> $\in (-180^\circ, +180^\circ]$

/LD150/	Speicherinhalt	GPX-Dokument (GpxResult)	
	Verweise	/LF200/	
	Attribute	gpxString	String UTF-8 (valides GPX 1.1)
		gpxBytes	byte[] UTF-8, kein BOM
		optimizationResult	OptimizationResult (Statistik)
		stats	SessionStats

/LD160/	Speicherinhalt	W3W-Cache-Eintrag	
	Verweise	/LF280/	
	Attribute	lat	double (gerundet 6 Dezimalstellen)
		lon	double (gerundet 6 Dezimalstellen)
		words	String (3 Wörter, Punkt-getrennt)
		resolvedAt	Instant UTC
		ttd	Duration (aus SessionConfig)

3.5.3 Testfälle (Traceability)

Die folgende Tabelle listet die geplanten Testfälle mit ihrem erwarteten Ergebnis und verknüpft jeden mit der Anforderung, die er absichert. Sie deckt sowohl die normalen Abläufe (Happy Path) als auch die Fehler- und Grenzfälle ab — etwa ungültige Koordinaten, erreichte Limits oder den abgewehrten Path-Traversal-Versuch.

TC-ID	Beschreibung	Erwartetes Ergebnis	LF / LL
TC-010	Doppelter <code>startSession()</code> -Aufruf.	<code>BearingStateException (ALREADY_ACTIVE)</code> .	/LF010/
TC-020	GpsFix mit <code>lat = 91°</code> .	<code>ValidationException (COORD_OUT_OF_BOUNDS)</code> .	/LF030/
TC-025	Zeitstempel +2h in der Zukunft.	<code>ValidationException (TIMESTAMP_TOO_FAR_IN_FUTURE)</code> .	/LF030/
TC-030	Update nach <code>complete()</code> .	<code>BearingStateException (SESSION_COMPLETED)</code> .	/LF030/
TC-040	Snapshot ohne Fix.	<code>Optional.empty()</code> .	/LF050/
TC-050	Soft-Limit (n=100).	<code>onSoftLimitReached</code> ; Aufzeichnung läuft.	/LF030/
TC-060	Hard-Limit STOP (n=200).	<code>onHardLimitReached</code> ; kein weiterer Punkt.	/LF030/

TC-ID	Beschreibung	Erwartetes Ergebnis	LF / LL
TC-070	<code>abort()</code> ohne <code>persist</code> .	GPX-String $\neq$ leer; keine Datei.	/LF070/
TC-080	<code>abort()</code> mit <code>persist</code> .	Datei atomar geschrieben; valide.	/LF070/
TC-090	Pfad <code>../../etc/passwd</code> .	<code>SecurityException</code> .	/LF200/
TC-100	GPX-Namespace in Ausgabe.	<code>xmlns="http://www.topografix.com/GPX/1/1"</code> .	/LF200/
TC-110	n-Optimizer n=10, 100 Punkte.	11 Punkte (Start/Ende + 9 dazwischen).	/LF200/
TC-120	Kollinear: 3 Punkte auf Linie.	2 Punkte nach Optimierung.	/LF200/
TC-130	Douglas-Peucker $\epsilon = 100$ m.	Starke Reduktion; kein Fehler.	/LF200/
TC-140	W3W-API Timeout.	Fallback; WARN-Log mit <code>sessionId</code> .	/LF280/
TC-150	W3W-Cache-Treffer (2. Aufruf).	Kein Netzwerkaufruf beim 2. Mal.	/LF280/
TC-160	Clock-Mock; zwei identische Läufe.	Byte-identischer GPX-String.	/LL190/
TC-170	Listener wirft <code>RuntimeException</code> .	Exception gefangen; nächster Listener weiter.	/LF080/
TC-180	<code>reset()</code> $\rightarrow$ neuer Start.	Neue UUID; Session ACTIVE.	/LL160/
TC-190	Sonderzeichen <code><>&</code> im Namen.	GPX enthält korrekte XML-Entities.	/LL130/
TC-200	Start Happy Path.	UUID vorhanden; <code>onSessionStarted</code> gefeuert.	/LF010/
TC-210	Azimut Stuttgart $\rightarrow$ München vs. Referenz.	Abweichung $\leq \pm 1^\circ$.	/LL020/
TC-220	Zeitlücke 6 min (Threshold 5 min).	Zwei <code><trkseg></code> im GPX.	/LF030/

Tabelle 15: Testfälle mit Traceability zu funktionalen und nicht-funktionalen Anforderungen.

3.5.4 FMEA

Die Fehlermöglichkeits- und Einflussanalyse (FMEA) betrachtet systematisch, was schiefgehen kann, welche Folge das hätte und wie die Bibliothek dem begegnet. Jede Zeile schätzt Auftretenswahrscheinlichkeit (A) und Entdeckungswahrscheinlichkeit bzw. Schwere der Folge (E) qualitativ ein und nennt die konkrete Gegenmaßnahme samt zugehöriger Anforderung.

Fehler	Folge	A	E	Gegenmaßnahme
Ungültige Koordinate	Falsche Peilung oder Exception.	M	K	ValidationException; /LF030/ Schritt 2.
Ungültiger Zeitstempel	Falsche Segmentierung.	M	M	Zeitvalidierung; /LF030/ Schritt 2.
Speicherverflow	OutOfMemoryError der JVM.	G	K	Hard-Limit /LF030/ Schritt 3 (Alt. 5a).
XML-Injection	Ungültige GPX-Datei.	G	M	XmlEscaper; /LF200/ Schritt 4.
Path Traversal	Kritische Datei überschreiben.	G	M	Pfadnormalisierung; /LF200/ Schritt 5.
W3W-Ausfall	Fehlende W3W-Adresse.	G	G	Fallback + Cache; /LF280/.
Listener wirft	Benachrichtigungsschleife abbricht.	M	K	Isolation-Garantie; /LF080/.
Clock nicht injiziert	Nicht-deterministische Tests.	S	G	ClockPort-Injection; /LL170/.

Tabelle 16: FMEA: Fehlermodi, Folgen, Klassen und Gegenmaßnahmen.

Legende A/E: S = selten, M = mittel, G = gering, K = katastrophal (qualitativ).

4 Anhang

Der Anhang bündelt vertiefende und unterstützende Artefakte, die den normativen Teil des Dokuments ergänzen, ohne ihn zu unterbrechen: erweiterte Akzeptanzkriterien, den Referenzalgorithmus der Peilung, die vollständige Traceability-Matrix, das objektorientierte Analyse- und Entwurfsmodell sowie Verzeichnisse, Checklisten und die Versionshistorie.

4.1 Erweiterte Akzeptanzkriterien (Kap. 3.1)

Die normativen Spezifikationen stehen in **Kapitel 3.1**. Dieser Abschnitt ergänzt die Varianten /LF280/, /LF230/, /LF080/ und /LL160/ um prüfbare Akzeptanzkriterien.

/LF / /LL	Trigger	Akzeptanzkriterien
/LF280/	W3W-Key fehlt	Fallback-String zurückgegeben, kein Netzwerkaufruf ausgelöst.
/LF280/	Netzwerk-Timeout W3W	WARN-Log mit <code>sessionId</code> , Fallback-Wert, kein Crash.
/LF230/	GPX nur als Bytes	<code>byte[]</code> UTF-8 ohne BOM; Länge > 0 nach abgeschlossener Session.
/LF080/	Listener wirft	Exception wird intern gefangen; Session bleibt in konsistentem Zustand.
/LL160/	Reset nach Ende	Zustand IDLE erreichbar nach ABORTED oder COMPLETED.

Tabelle 17: Erweiterte Akzeptanzkriterien zu ausgewählten /LF.../- und /LL.../-Spezifikationen.

4.2 Haversine- und Azimutberechnung (Referenzalgorithmus)

Die folgende Beschreibung des Referenzalgorithmus dient der Übereinstimmung mit /LL020/ (Referenzabweichung $\leq \pm 1^\circ$). Die Implementierung nutzt `double` und muss Grenzfälle (identische Punkte, Polnähe) explizit absichern.

Eingabe: Breite/Länge von Standort φ_1, λ_1 und Ziel φ_2, λ_2 (in Radiant für die trigonometrischen Funktionen).

Haversine-Distanzberechnung:

$$\Delta\varphi = \varphi_2 - \varphi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1$$

$$a = \sin^2\left(\Delta\frac{\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\Delta\frac{\lambda}{2}\right)$$

$$d = 2R \cdot \arcsin(\sqrt{a})$$

wobei

$$R = 6.371.000$$

m der mittlere Erdradius ist.

Azimutberechnung:

$$\alpha = \text{atan2}(\sin(\Delta\lambda) \cdot \cos(\varphi_2), \cos(\varphi_1) \cdot \sin(\varphi_2) - \sin(\varphi_1) \cdot \cos(\varphi_2) \cdot \cos(\Delta\lambda))$$

Anschließend Normalisierung auf

$$[0^\circ, 360^\circ)$$

: `azimuth = (alpha + 360°) mod 360°`.

Grenzfälle:

- Identische Punkte ($d = 0$): Azimut undefiniert $\rightarrow$ Rückgabe 0° per Konvention.
- Polnähe: Numerische Instabilität durch $\cos(\varphi) \rightarrow 0$ muss durch Klammertests abgesichert werden.
- Antipodische Punkte ($d \approx \pi R$): Azimut mehrdeutig $\rightarrow$ dokumentieren und testen.

4.3 Douglas–Peucker auf Kugeloberfläche (Hinweis für Erweiterungen)

Klassisches Douglas–Peucker arbeitet in der Ebene. Für lange Tracks auf der Erdkugel sollte die Metrik entweder in **lokaler Tangentialebene** (ECEF-Projektion pro Segment) oder über **Chordal-Distanzen** approximiert werden.

Die Spezifikation /LF270/ verlangt eine **metrische Toleranz in Metern** (Epsilon). Die Implementierung muss daher dokumentieren, welche Näherungsmetrik verwendet wird:

- **Lokale Tangentialebene (empfohlen für kurze Segmente):** ECEF-Projektion des Mittelpunkts; gültig für Segmentlängen < 50 km.
- **Chordal-Distanz:** Euklidische Distanz im 3D-Raum als Näherung für die geodätische Querabweichung.

Parameter im Code: `epsilonM` (in Metern). Der Start- und Endpunkt eines Segments werden immer erhalten.

4.4 Vollständige Traceability-Matrix

Die folgende Matrix stellt die vollständige Rückverfolgbarkeit von Anforderungen über OOA-Konzepte und OOD-Klassen bis zu Testclustern her.

/LF	OOA-Konzept	OOD-Klasse	Test-Cluster
/LF010/	Peilung	BearingSession	TC-200, TC-010
/LF030/	Standortfolge	BearingCalculator, GpsFix	TC-020, TC-210
/LF050/	Peilungsgrößen	BearingSnapshot	TC-210
/LF060/	Export	GpxSerializer, BearingSession	TC-GPX-01..03
/LF070/	Peilung	BearingSession	TC-070, TC-080
/LF080/	Peilung	BearingListener	TC-011
/LF100/	Standortfolge	TrackAggregator	TC-050, TC-060
/LF120/	Standortfolge	TrackAggregator	TC-050, TC-060
/LF170/	Track	TrackAggregator	TC-TRACK-04
/LF180/	Track	TrackAggregator	TC-060
/LF200/	Export	GpxSerializer	TC-100
/LF230/	Export	GpxResult	TC-100, TC-GPX-01..03
/LF220/	Export	SafeFileSink	TC-080
/LF240/	Trackoptimierung	NthPointOptimizer	TC-110
/LF260/	Trackoptimierung	CollinearityOptimizer	TC-120
/LF270/	Trackoptimierung	DouglasPeuckerOptimizer	TC-130
/LF280/	Externe Referenz	W3wAdapter	TC-W3W-01..03
/LF290/	Externes Caching	W3wCacheAdapter	TC-150
/LF320/	Export	SafeFileSink	TC-090
/LF340/	Peilung	ClockProvider	TC-160
/LF400/	Peilung	SessionConfig, BearingSession	TC-400
/LL160/	Peilung	BearingSession	TC-180

Tabelle 18: Vollständige Traceability-Matrix: Anforderung → OOA-Konzept → OOD-Klasse → Test.

4.5 Objektorientiertes Analyse- und Entwurfsmodell (OOA/OOD)

Dieser Abschnitt trennt bewusst zwei Sichten: Die objektorientierte Analyse (OOA) hält fest, *was* die Domäne ausmacht; der objektorientierte Entwurf (OOD) legt fest, *wie* sie technisch umgesetzt wird. Diese Reihenfolge stellt sicher, dass der Entwurf der Fachlichkeit folgt und nicht umgekehrt.

4.5.1 OOA: Domänenmodell

Die Analyse betrachtet die Fachlichkeit, bevor technische Entscheidungen einfließen. Sie hält fest, welche Konzepte die Domäne kennt und wie sie zusammenhängen — noch ohne Klassen, Schnittstellen oder Frameworks. Die Assoziationen sind daher fachlich zu lesen, die Multiplizitäten bewusst pragmatisch gehalten. Das folgende Domänenmodell bildet die Grundlage für den technischen Entwurf im nächsten Abschnitt.

Konzept	Beschreibung	Multiplizität
Peilung	Fachlicher Vorgang vom Start bis Complete/Abort.	1 pro Session
Zielpunkt	Geo-Koordinate (WGS84).	1
Standortfolge	Zeitlich geordnete GPS-Fixes.	0..*
Peilungsgrößen	Abgeleitete Werte (Azimut, Distanz, Ordinal).	0..1 vor erstem Fix
Track	Sammlung von Segmenten.	0..1
Export	GPX-Dokument als Artefakt.	0..1 nach Abschluss

Tabelle 19: Fachliches Domänenmodell (OOA): Konzepte und ihre Beziehungen.

Assoziationen: Peilung hat genau einen Zielpunkt und genau einen Track. Der Track enthält beliebig viele Trackpunkte. Die Peilung kann optional einen Export (GPX) erzeugen. Standortfolge: 0..*.

4.5.2 OOD: Technischer Feinentwurf

Der Entwurf überführt die fachlichen Konzepte in konkrete Klassen und Schnittstellen und ordnet ihnen ein Stereotyp zu — Value Object, Domain Service, Infrastructure oder Port. Auffällig ist die durchgängige Trennung über Port-Interfaces (`TrackOptimizer`, `W3wClient`, `ClockProvider`): Sie hält die Domäne frei von technischen Abhängigkeiten und erlaubt, im Test echte Implementierungen durch Mocks zu ersetzen.

Klasse / Interface	Stereotyp	Kernverantwortung
BearingSession	Entity/Controller	Zustandsautomat IDLE/ACTIVE/COMPLETED/ABORTED.
SessionConfig	Value Object	Immutable Konfiguration; Builder-Pattern.

GeoCoordinate	Value Object	WGS84 lat/lon validiert.
GpsFix	Value Object	Zeit + Position + optional HDOP/Speed/Elevation.
BearingCalculator	Domain Service	Haversine-Distanz, Azimut, Ordinalrichtung.
TrackAggregator	Domain Service	Rohspeicher, Segmente, Punktbudget.
GpxSerializer	Infra- structure	GPX 1.1, XML-Escaping, UTF-8.
SafeFileSink	Infra- structure	Atomares Dateischreiben, Path-Traversal-Schutz.
TrackOptimizer	Interface (Stra- tegy)	Optimierungsstrategie austauschbar.
W3wClient	Interface (Port)	Reverse-Lookup-Abstraktion.
ClockProvider	Interface (Port)	Testbarkeit via Clock-Injection.
BearingListener	Interface (Obser- ver)	Ereignisbenachrichtigung für den Host.
NthPointOptimizer	Domain Service	Jeder n-te Punkt; Start/Ende erhalten (/LF240/).
MinDistOptimizer	Domain Service	Mindestabstand in Metern (/LF250/).
CollinearityOptimizer	Domain Service	Kollinearitätsreduktion (/LF260/).
DouglasPeuckerOptimizer	Domain Service	Douglas-Peucker mit metrischem Epsilon (/LF270/).

Tabelle 20: OOD-Klassenübersicht: Zentrale Entwurfsklassen und Schnittstellen.

4.5.3 Zustandsdiagramm: Session-Lebenszyklus

Zustände: IDLE → ACTIVE (start) → COMPLETED (complete) oder ABORTED (abort). Übergänge aus COMPLETED/ABORTED zurück zu IDLE durch explizite reset()-Operation (/LL160/).

Invariante ACTIVE: Positionsupdates zulässig; Konfiguration fix; UUID vergeben.

Invariante COMPLETED/ABORTED: Keine weiteren Positionsupdates (/LF060/, /LF070/).

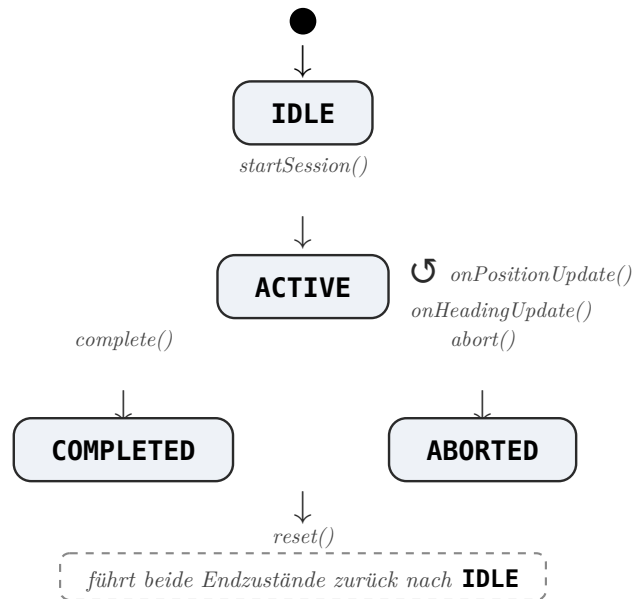


Abbildung 16: Zustandsdiagramm des Session-Lebenszyklus.

Die folgende Tabelle führt dieselben Übergänge mit den jeweiligen Guards und Aktionen im Detail auf.

Von	Nach	Auslöser	Guard / Aktion
IDLE	ACTIVE	startSession()	Konfiguration gültig; UUID erzeugt; onSessionStarted gefeuert.
ACTIVE	COMPLETED	complete()	GPX materialisiert; onSessionCompleted gefeuert.
ACTIVE	ABORTED	abort()	GPX materialisiert; optional persistiert; onSessionAborted gefeuert.
COMPLETED	IDLE	reset()	Alle internen Zustände gelöscht.
ABORTED	IDLE	reset()	Alle internen Zustände gelöscht.
ACTIVE	ACTIVE	onPositionUpdate()	Fix validiert, gespeichert, Peilung berechnet, Listener benachrichtigt.

Tabelle 21: Zustandsdiagramm: Session-Lebenszyklus mit Transitionen und Guards.

4.5.4 Subsystem-Dekomposition

Erster Schritt – Subsystem-Identifikation:

Dienst	Verhalten	Fehlerfälle
Session-Management	Steuert den Lebenszyklus einer Peilung und liefert eine Referenz-UUID zurück.	Peilung bereits aktiv; Zielkoordinaten ungültig.
Tracking-Logik	Wendet Validierungsvorschriften auf GPS-Daten an und berechnet Peilungsfortschritte.	Ungültige Koordinaten; Zeitstempel außerhalb Bereich.

Kurs-Analyse	Berechnet Kursabweichungen basierend auf Azimut und Heading.	Kurswert außerhalb $[0^\circ, 360^\circ]$; fehlende Datenbasis.
Export-Dienst	Koordiniert Finalisierung und GPX-Serialisierung; optional Dateischreiben.	Session nicht abgeschlossen; Serialisierungsfehler.
Optimierungs-Dienst	Führt konfigurierte Optimizer-Strategien vor dem Export sequentiell aus.	Konfigurationsfehler; leere Punktliste.
W3W-Look-up	Kapselt HTTP-Anfragen und Cache-Zugriffe für Reverse-Lookup.	API nicht erreichbar; kein API-Key; Cache-Miss + Netzwerkfehler.

Tabelle 22: Subsystem-Identifikation: Dienste und ihre Fehlerfälle.

Zweiter Schritt – Subsystem-Anordnung:

Die Anordnung folgt einer streng hierarchischen Struktur (Presentation Layer → Service Layer → Business Rules Layer → Data Access Layer). Die Kommunikation erfolgt strikt einseitig von oben nach unten; direkter Zugriff von der API-Ebene auf den Data Access Layer ist verboten.

4.6 Vorlesungs-Themen-Matrix (Abdeckung SWE)

Die folgende Matrix ordnet die Themengebiete der Softwareengineering-Vorlesung den Dokumentations- und Implementationsartefakten dieses Projekts zu.

Themenfeld	Projektbezug	Artefakt / Ort
Requirements Engineering	SOPHIST-Regeln, /LF//LL//LD, Traceability	Kap. 3; Matrizen
Lastenheft/Pflichtenheft	Zielhierarchie, Spezifikation nach IEEE 830	Kap. 2, Kap. 3
Use-Case-Modellierung	/LF.../-Spezifikationen mit Aktivitätsdiagrammen	Kap. 3.1, Anhang
Aktivitätsdiagramme	Positionsupdate, GPX-Export, Session-Lebenszyklus	Kap. 3.1
Sequenzdiagramme	complete()-Pfad, Fehlerpfad Validierung	Kap. 3.3
Zustandsdiagramme	Session-Lebenszyklus IDLE/ACTIVE/COMPLETED/ABORTED	Anhang A.3
OOA/OOD	Domänenmodell, Klassentabelle, Entwurfsklassen	Anhang A.3
Entwurfsmuster	Strategy (Optimizer), Observer (Listener), Builder (Config), Port	Kap. 2, Kap. 3
Schichtenarchitektur	API, Domain Core, Ports, Infrastruktur	Kap. 2.1
Qualitätsmodelle	ISO/IEC 25010 Gewichtungsmatrix	Kap. 3.5
Nicht-funktionale Anforderungen	Performance, Sicherheit, Portabilität (/LL.../)	Kap. 3.2
Testen	JUnit 5, JaCoCo, FMEA, Testfälle	Kap. 3.5
Konfigurationsmanagement	Maven, Profile w3w	Kap. 2.5, / LF450/
Risikomanagement	FMEA-Tabelle, Sicherheitsanforderungsklassen	Kap. 2.4, Kap. 3.5
Dokumentationsstandards	IEEE/ISO Bezug, Javadoc-Pflicht, SOPHIST	Kap. 1.1, / LL100/
Sicherheit	Path Traversal, XML-Escaping, XXE-Schutz	Kap. 2.4, / LF320/
Performance Engineering	Mikrobenchmark-Vorgaben, Heap-Limit	Kap. 3.4, / LL030/

Tabelle 23: Vorlesungsthemen und ihre Abdeckung durch Projektartefakte.

4.7 Literatur- und Normenverweise

Quelle	Kontext im Projekt
ISO/IEC/IEEE 29148:2018	Requirements Engineering, Traceability
ISO/IEC 25010:2011	Produktqualitätsmodell (Qualitätsmatrix Kap. 3.5)
IEEE 830-1998	Software Requirements Specifications (Dokumentaufbau)
GPX 1.1 Schema	XML-Export, Namespace http://www.topografix.com/GPX/1/1
WGS84 / EPSG:4326	Koordinatenreferenzsystem für alle Geo-Eingaben
What3words API Docs	Optionales Reverse-Lookup-Interface
OWASP	Path Traversal, XML-Sicherheitsrichtlinien
Gamma et al.: Design Patterns (GoF)	Strategy, Observer, Builder, Factory
Bloch: Effective Java	Immutability, Exceptions, API-Design
JUnit 5 User Guide	Teststrategie und Testframework
SLF4J Manual	Beobachtbarkeit und strukturiertes Logging
Douglas, Peucker (1973)	Algorithmus zur Linienvereinfachung /LF270/
SOPHIST-Methode	Anforderungsformulierung nach SOPHIST-Regeln
ISO 8601	Zeitstempel-Format UTC; <code>DateTimeFormatter.ISO_INSTANT</code>
Apache Maven Docs	Build-System, Profilverwaltung (w3w-Profil)

Tabelle 24: Primäre Normreferenzen und technische Quellen des Projekts.

4.8 Glossar-Index (Schlagwort → Abschnitt)

Begriff	Vorkommen
Peilung	Glossar (Kap. 1.3); /LF010/-/LF050/
Azimut	Glossar; /LF050/; Anhang A.2
Haversine-Formel	Glossar; /LL020/; Anhang A.2
GPX 1.1	Glossar; /LF200/; Kap. 3.3; Anhang A.2
Session	Glossar; /LF010/-/LF090/; Anhang A.3
SOPHIST	Glossar; Kap. 1.1; Kap. 3 (Einleitung)
Strategy-Pattern	Glossar (Trackoptimierung); /LF480/; Anhang A.3
Observer-Pattern	Glossar (Listener); /LF080/; Kap. 3.3
Builder-Pattern	Glossar; /LF410/
Immutability	Glossar; /LF410/; Kap. 2.2
Douglas-Peucker	Glossar; /LF270/; Anhang A.2
Path Traversal	Glossar; /LF320/; Kap. 2.4
XXE	Glossar; /LL130/; Kap. 2.4
Atomares Schreiben	Glossar; /LF220/; Kap. 3.3
Soft-Limit / Hard-Limit	Glossar; /LF120/, /LF180/
What3Words	Glossar; /LF280/, /LF290/
SLF4J	Glossar; /LL140/
JaCoCo	Glossar; /LL150/
Maven	Glossar; /LF450/; Kap. 2.5
Traceability	Glossar; Anhang A.1; Kap. 3.5

Tabelle 25: Glossar-Schnellreferenz: Begriff und Fundstelle im Dokument.

4.9 Review-Checkliste „SOPHIST-Kriterien,, (Selbstaudit)

Die folgende Checkliste dient dem Selbstaudit vor der Abgabe. Alle Punkte müssen erfüllt sein.

1. Keine User Stories als Ersatz für normierte SOPHIST-Anforderungen verwendet.
2. Alle Anforderungen sichtbar und eindeutig identifiziert: /LF, /LL, /LD durchgängig.
3. Spezifikation präziser als Analyse: /LF.../-Spezifikationen, Zustandsdiagramm, Sequenzen vorhanden.
4. Objektorientierte Analyse: Domänenmodell separat von technischen Entwurfsklassen.
5. Aktivitätsdiagramm-Material vorhanden (pro /LF.../ in Kap. 3.1).
6. Testfälle abgegeben, nachvollziehbar benannt (TC-Nummern).
7. Quellcode per Maven (`mvn test`) ohne manuelle Schritte ausführbar.
8. Keine UI-Abhängigkeiten in der Bibliothek (/LF430/).
9. Abbruch (`abort()`) liefert GPX-Daten, nicht zwingend Datei (/LF070/, /LF230/).
10. Javadoc vollständig für alle öffentlichen API-Methoden (/LL100/).
11. Sicherheitsanforderungen adressiert: Path Traversal (/LF320/), XML-Escaping (/LL130/).
12. Traceability-Matrix verknüpft Anforderungen mit Klassen und Testfällen.

4.10 Versionshistorie

Version	Datum	Autor/in	Änderungen
0.1	April 2026	Pfitzenmaier, Müllmaier, Bensch	Initiale Gliederung und Anforderungserhebung.
0.5	Mai 2026	Pfitzenmaier, Müllmaier, Bensch	Vervollständigung Glossar, Anforderungskarten, Grobentwurf.
1.0	Juni 2026	Pfitzenmaier, Müllmaier, Bensch	Finale Überarbeitung; neues Inhaltsverzeichnis nach IEEE 830; Kapitelstruktur 1/2/3 umgesetzt; Qualitätssicherung finalisiert.

Tabelle 26: Versionshistorie des Dokuments.

Werkzeugkette: Typst-Quelle (main.typ) erzeugt PDF; Maven erzeugt Java-Artefakte.

Norm-Referenz: Dokumentstruktur nach IEEE 830 / ISO/IEC/IEEE 29148.

